

Informe comentado del do-file: selector de diseño/balance para ATET con teffects ipwra

Daniel

20 de febrero de 2026

Índice

1. Cómo leer este informe	1
2. Encabezado y configuración inicial	1
3. 0) Rutas y carpetas de salida	4
4. 1) Parámetros globales y tolerancias	4
5. 2) Carga de datos y definición de panel	7
6. 2.4) Switches de especificación (qué entra en X)	7
7. 2.5) Transformaciones y FE (variables explícitas)	9
8. 5) Covariables candidatas del tratamiento	13
9. 5.b) Covariables del modelo de outcome	14
10.3) Restricción central y construcción del outcome	15
11.4) Construcción del tratamiento y auxiliares de organización	16
12.Programa build_xlist: armar listas de covariables con/sin rezagos	18
13.5.c) Construcción de especificaciones S1–S5	20
14.Programa eval_design: proxy IPW manual ($mlogit + w=p_t/p_0$)	22
15.Programa eval_design_tew: diseño con trimming y pesos tipo teffects	26
16.Programa eval_design_teb: benchmark multinomial y métricas de balance	32

17.7) Ejecución del selector (loops por modo y spec)	45
18.7.1) Exportación de tablas largas	48
19.7.2) Resumen agregado por (modo, spec)	49
20.7.3) Checks de consistencia entre TEW y TEB	51
21.7.4) Ranking y elección de ganadores	52
22.8) Estimación final ATET y gráficos (overlap + hist de pesos)	55
23. Resumen ejecutivo del do-file	60

1. Cómo leer este informe

Este informe está organizado por bloques. En cada sección se copia un tramo del do-file y debajo se explica qué hace y cómo encaja en el flujo completo.

Si querés una ruta rápida de lectura: (i) empezá por los parámetros globales y reglas de trimming (4); (ii) leé el evaluador benchmark que alimenta el ranking (16); (iii) mirá cómo se ejecuta el selector y cómo se guardan los resultados (17 y 18); (iv) terminá en el ranking y selección de ganadores (21); (v) si activás el módulo opcional, el bloque final corre la estimación y genera gráficos (22).

2. Encabezado y configuración inicial

Código (líneas 1–55)

```

/*-----
3ATET_IPWRA_design_balance_selector_tebalance_v19.do
Autor: Daniel + ChatGPT
Fecha: 2026-02-17

OBJETIVO (versión “operativa” y sin episodios; con GLOBALS como preferís):

(1) MUESTRA:
    - Trabajar SOLO dentro de b1_9==1 (empresas con actividad de innovación).
    - Outcome (Y_cond): mejora ambiental (1 si resul_mejora_ambiente in {1,2})
      definido SOLO entre innovadores.
    -> NO se convierte a 0 para no innovadores; fuera de b1_9==1 el outcome no
      está definido.

(2) TRATAMIENTO (estado, NO episodios):
    D_pp = 0: No prod/proc
           1: Solo producto
           2: Solo proceso

```

```

3: Ambas
(NO por episodios: por diseño te deja celdas minúsculas y rompe el solapamiento
.)

(3) DOS MODOS respecto a “innovación organizacional/comercial” (
innova_organiz_comerc):
MODO 1) IGNORE_ORG:
    - NO se excluye org al definir D_pp (se usa toda la muestra de innovadores).

    - Opcional: controlar por org con auxiliares org1 y org_miss.
MODO 2) STRICT_AVOID_ORG:
    - Excluir observaciones con org==1 y también con org missing.
    - (Más “limpio” si tu objetivo es aislar prod/proc sin contaminar por org,
      pero reduce MUCHO la muestra y puede romper mlogit/overlap.)

(4) DISEÑO “A CIEGAS” (SIN mirar el ATET):
    - Se comparan pocas especificaciones candidatas (sets de X) con métricas de
      balance/solapamiento:
        max|SMD|, mean|SMD|, max|log(var ratio)|, ESS0, N1, N0, concentración de
        pesos (p95/p99/max).
    - Se elige una especificación “ganadora” por modo usando SOLO esas métricas.
    - Recién después (si RUN_ATET=1) se corre teffects ipwra y se exportan gráficos
      de overlap/pesos.

NOTA CLAVE SOBRE org1 y org_miss (cuando INCLUDE_ORG_CONTROLS=1 en IGNORE_ORG):
    - org1: dummy (org==1), rellenando missing como 0 para no perder muestra.
    - org_miss: dummy de missing en org (separa “0 real” de “no observado”).
    - En IPWRA podés usar covariables distintas en el modelo de tratamiento y el de
      outcome.
    Por defecto acá, si incluís org controls, podés elegir:
        ORG_IN_YMODEL=1 -> org1 y org_miss van también al outcome model.
        ORG_IN_YMODEL=0 -> org1 y org_miss SOLO van al tratamiento model.
    (No hay “una verdad única”: 1 suele estabilizar; 0 es más conservador si tem
    és bad controls.)

IMPORTANTE:
    - Si cambiás locals a globals, NO uses "=" en globals numéricos.
    Correcto: global RUN_ATET 1
    Incorrecto: global RUN_ATET = 1    (eso guarda el string "= 1" y rompe ifs).
-----*/

version 19.0
clear all
set more off

*=====

```

Explicación

Este do-file implementa un pipeline completo para seleccionar, de forma reproducible, una especificación de covariables (**X**) que produzca un diseño razonable para estimar efectos de tratamiento promedio sobre los tratados (ATET) con `teffects ipwra` cuando el tratamiento es multivaluado ($D \in \{0, 1, 2, 3\}$). En términos prácticos:

(1) construye `Y_cond` (outcome binario) y dos versiones del tratamiento (`D_pp_ignore` y `D_pp_strict`); (2) arma un conjunto de especificaciones candidatas **S1-S5** para el modelo de selección (el `mlogit` del tratamiento); (3) evalúa cada (*modo*, *spec*, *t*) con métricas de soporte/overlap, concentración de pesos y balance de covariables; y (4) rankea y elige “ganadores” por modo, dejando todo listo para correr la estimación final y generar gráficos diagnósticos.

A lo largo del informe voy a respetar la lógica del archivo: pego bloques de código y debajo explico qué hacen y por qué están ahí. Cuando aparece una métrica (p.ej. `ESS0` o `maxSMD_w`) voy a explicitar: (i) cómo se calcula, (ii) qué intenta medir, y (iii) en qué bloque del código se construye.

Glosario rápido de métricas que aparecen repetidamente

Comparación *t*: el archivo evalúa tres comparaciones binarias *t* vs 0 (con *t*=1,2,3), donde 0 es el grupo base y *t* uno de los estados tratados.

Pesos ATET (controles): en el enfoque IPW para ATET, los controles se re-ponderan como $w_i = \Pr(D = t \mid X_i) / \Pr(D = 0 \mid X_i)$. En el código esto aparece como `w_raw = p‘t’/p0`. Los tratados llevan peso 1.

N1, N0: tamaños muestrales finales de tratados y controles *después* de aplicar las reglas de recorte (missing en **X**, pisos de `p0`, trims por `p0` y/o por cuantiles de pesos, y techo duro `W_CEIL`).

ESS0: “effective sample size” en controles bajo ponderación IPW. Se calcula como $(\sum w)^2 / \sum w^2$ y baja cuando los pesos están muy concentrados.

maxSMD, meanSMD: máximo y promedio del *absolute standardized mean difference* (SMD) de las covariables entre tratados y controles ponderados. Captura desbalance en medias.

maxLogVarRatio: máximo de $|\log(\text{Var}_w(X \mid D = 0) / \text{Var}(X \mid D = t))|$. Captura desbalance en dispersión de covariables; el uso de $|\log(\cdot)|$ lo vuelve simétrico (VR=2 y VR=0.5 penalizan igual).

w_p95, w_p99, w_max: percentiles y máximo de pesos (controles). Sirven para monitorear outliers de ponderación.

share_trim / share_drop *: fracciones eliminadas o afectadas por cada regla (piso de `p0` en controles, piso de `p0` en tratados, trimming por rango de `p0`, techo duro de pesos, etc.). Estas métricas son las que te dicen “qué tan agresivo” fue el filtro.

p0_*_min/max: rangos de $\Pr(D = 0 \mid X)$ por grupo (tratados vs controles) que ayudan a ver overlap en soporte.

3. 0) Rutas y carpetas de salida

Código (líneas 56–73)

```

* 0) Rutas (GLOBAL)
*=====
global seteo "C:\Users\Equipo\OneDrive\Desktop\proyecto\4Medio ambiente- innovacion
\"

local username = c(username)
if "'username'"=="Equipo" {
    global seteo "C:\Users\Equipo\OneDrive\Desktop\proyecto\4Medio ambiente-
    innovacion\"
}
else if "'username'"=="dmendez" {
    global seteo "C:\Users\dmendez\OneDrive\Desktop\proyecto\4Medio ambiente-
    innovacion\"
}

* Carpeta de salida
global outdir "${seteo}3. Estrategia empirica\3. Modelos\5Resultado-innovacion -
    estrategia causal\_outputs_ipwra"
cap mkdir "${outdir}"

*=====

```

Explicación

Este bloque define rutas y crea (si no existe) una carpeta de salida. Hay dos ideas prácticas:

Primero, el do-file intenta ser portable: arma la raíz `seteo` en función del usuario de Windows (`c(username)`). Esto evita tener que editar manualmente el path cada vez que corrés el script en otra máquina/perfil.

Segundo, centraliza los outputs en `outdir`. Todo lo que sea tablas (`.xlsx`), bases intermedias (`.dta`) y gráficos (si los activás) debería idealmente caer ahí. Más adelante hay un `log using` con un path absoluto distinto; eso funciona, pero rompe un poco la idea de portabilidad (lo marco cuando lleguemos a ese bloque).

4. 1) Parámetros globales y tolerancias

Código (líneas 74–132)

```

* 1) Parámetros (GLOBAL)
*=====
* 1=usar lags (L1_) cuando existan; 0=usar contemporáneos
global USE_LAGS 0

* En modo IGNORE_ORG: ¿agregar org1 + org_miss como covariables?

```

```

global INCLUDE_ORG_CONTROLS 1

* Si INCLUDE_ORG_CONTROLS=1: ¿org entra también en el outcome model?
* 1 = sí (treatment y outcome), 0 = solo treatment model
global ORG_IN_YMODEL 1

* ¿Correr ATET + gráficos al final con la spec ganadora?
global RUN_ATET 1

* Umbrales prácticos para warnings (no son "reglas")
global MIN_N_USED 300
global MIN_N1 30
global MIN_NO 50

* Para evitar explosiones numéricas por  $p_0 \sim 0$ 
global PO_FLOOR 1e-8
global W_CEIL 1e6

* -- Mejoras selector v8: overlap y trimming robusto --
* Para ATET, el solapamiento crítico es que  $p_0(X) > 0$  también entre tratados (evita "
    tratados sin contrafactual").
global PO_TREAT_FLOOR 1e-8

* Trimming de pesos de controles ( $w = p_t/p_0$ ): CAP (winsoriza) o DROP (elimina)
global W_TRIM_STYLE CAP

* Cuantil para cap/drop (p.ej. 0.99 o 0.995)
global W_CAP_Q 0.99

* Trimming adicional explícito sobre  $p_0$  (probabilidad de ser control) si querés una
    regla tipo  $e(X) \in [a,b]$ 
* 0=off (default); 1=on
global PS_TRIM 0
global PS0_LOW 0.01
global PS0_HIGH 0.99

* Parsing de tebalance: 1=estricto (no usa fallback silencioso), 0=usa fallback con
    warning
global STRICT_TEB_PARSE 1

* Debug: imprimir colnames/validaciones de tebalance una sola vez
global DEBUG_TEB 0
global TEB_PRINTED 0

* Control de ejecución (por defecto: benchmark multinomial ON; proxy IPW (tew) OFF;
    fallback binario OFF)
global RUN_TEB 1
global RUN_TEW 0

```

```

global RUN_BIN_BENCH 0
* Mostrar tablas en consola (sin abrir .dta) al final del bloque 7
global SHOW_LONG_TABLES 1

* Tolerancia de balance para selector (tebalance): max|SMD| <= SMD_TOL
global SMD_TOL      0.10

*=====

```

Explicación

Acá se fijan “perillas” globales que controlan tres cosas: (i) qué tan exigente es el filtro de soporte y trimming, (ii) qué módulos del pipeline se ejecutan, y (iii) cómo se rankean las especificaciones.

Puntos clave:

Ejecución (qué corre). RUN_TEB controla si se ejecuta el evaluador principal `eval_design_teb` (benchmark multinomial). RUN_TEW controla el evaluador alternativo `eval_design_tew`. En esta versión, RUN_TEB=1 y RUN_TEW=0, por lo que el ranking se construye con las métricas de Sección 16.

Tamaño mínimo. MIN_N_USED impone un umbral mínimo de observaciones luego de la eliminación listwise por missing en covariables. Evita que el selector elija specs que “funcionan” solo porque quedan 30 observaciones.

Reglas de soporte/overlap. PO_FLOOR y PO_TREAT_FLOOR son pisos para $\Pr(D = 0 \mid X)$. Intuición: si $\Pr(D = 0 \mid X)$ es muy chica, el peso p_t/p_0 explota. El piso en tratados (PO_TREAT_FLOOR) agrega además una noción de soporte: si un tratado tiene prácticamente probabilidad cero de pertenecer al grupo base, está fuera de la región de comparación razonable.

Techo duro de pesos. W_CEIL es un “cinturón de seguridad”: descarta controles con w extremadamente alto (o no definido). Esto previene que una sola observación domine el pseudo-control.

Trimming por cuantiles. W_CAP_Q define el cuantil superior que se usa para recortar pesos en controles. Con W_TRIM_STYLE="CAP" se winsoriza (se reemplazan pesos mayores al cuantil por el valor del cuantil). Con "DROP" se eliminan. La winsorización mantiene más muestra, pero puede seguir dejando colas pesadas; el drop reduce colas pero cambia la composición.

Trimming por rango de p_0 . Si PS_TRIM=1, se eliminan observaciones (tratadas y controles) con p_0 fuera de $[PS0_LOW, PS0_HIGH]$. Esto es un filtro directo de overlap, independiente de los pesos.

Tolerancias de balance. SMD_TOL aparece como tolerancia “de referencia” (p.ej. para diagnósticos). Ojo: el ranking final usa umbrales explícitos en el bloque Sección 21 (ahí se ve la regla exacta).

En resumen, este bloque define el “contrato” de qué es un diseño aceptable: buen overlap, pesos no explosivos, y balance razonable.

5. 2) Carga de datos y definición de panel

Código (líneas 133–140)

```
* 2) Base y panel (delta 3)
*=====
use "${seteo}3. Estrategia empirica\3. Modelos\panel_innovacion_ipi_8.dta", clear

isid correlativo anio
xtset correlativo anio, delta(3)

*=====
```

Explicación

Se carga la base `panel_innovacion_ipi_8.dta` y se declara estructura de panel con `xtset correlativo anio, delta(3)`. Eso tiene dos roles:

- (1) **Consistencia del panel.** `isid correlativo anio` fuerza unicidad a nivel individuo-año, lo cual es crítico si después se generan rezagos (`L1.`) u otras operaciones de series.
- (2) **Rezagos trienales.** `delta(3)` indica que el salto temporal entre ondas es 3, por lo que `L1.x` representa el valor de la onda previa (tres años antes), no un año antes.

Este seteo es la base de todo lo que viene en el armado de rezagos en Sección 9 y en el helper Sección 12.

6. 2.4) Switches de especificación (qué entra en X)

Código (líneas 141–199)

```
* 2.4) Control de especificación de covariables (switches)
*=====
* Objetivo: poder prender/apagar, SIN tocar el ‘motor’ del selector:
*   - Cuadráticos (tamaño, edad)
*   - Interacciones (redes$\times$expor, redes$\times$tamaño)
*   - FE por ola (anio) y FE por sector (industry_id)
*
* Nota: NO usamos c./i. (factor vars) porque este pipeline trabaja con listas
* de nombres de variables existentes + checks numéricos (confirm numeric).
* La forma ‘limpia’ acá es crear variables explícitas (numéricas) y luego
* decidir si entran o no en cada ecuación.

* --- Qué entra en el MLOGIT del tratamiento (D) ---
```



```

* si use LAG 0
global TR_USE_SIZE_SQ      0
global TR_USE_AGE_SQ      0
global TR_USE_INT_RXEXP   1    // quiza desactivar
global TR_USE_INT_RXSIZE  1
global TR_USE_FE_WAVE     0    // si se activa se rompe en el modo estrico. en
    tanto que, si bien contemporaneo no rompe, las dummies de ola te empeoran mucho
    las metricas de diseño.
global TR_USE_FE_SECTOR   1

* --- Qué entra en el modelo de outcome (Y) ---
global Y_USE_SIZE_SQ      1
global Y_USE_AGE_SQ      1
global Y_USE_INT_RXEXP   1
global Y_USE_INT_RXSIZE  1
global Y_USE_FE_WAVE     0    // si se activa se rompe en el modo estrico. en tanto
    que, si bien contemporaneo no rompe, las dummies de ola te empeoran mucho las
    metricas de diseño.
global Y_USE_FE_SECTOR   1

* si use LAG 1
/*
* --- Qué entra en el MLOGIT del tratamiento (D) ---
global TR_USE_SIZE_SQ      0
global TR_USE_AGE_SQ      0
global TR_USE_INT_RXEXP   1
global TR_USE_INT_RXSIZE  1
global TR_USE_FE_WAVE     0    // en lag, se rompe si se activa en ambos modos.
global TR_USE_FE_SECTOR   1
* --- Qué entra en el modelo de outcome (Y) ---
global Y_USE_SIZE_SQ      1    // en lag, se rompe el modo estricto.
global Y_USE_AGE_SQ      0
global Y_USE_INT_RXEXP   1    // en lag, se rompe el modo estricto. (podria
    dejarla activada si solo nos basamos en el modo "ignoro", siempre que no
    empeore metricas)
global Y_USE_INT_RXSIZE  1
global Y_USE_FE_WAVE     0    // en lag, se rompe si se activa en ambos modos.
global Y_USE_FE_SECTOR   1
*/

* Límite de categorías para sector FE (evita romper mlogit por separación)
global MAX_SECTOR_FE 12

* Debug de armado de listas (1 imprime listas finales; 0 silencioso)

```

```
global DEBUG_XSETS 0
```

```
*=====
```

Explicación

Acá se define *qué entra* en los modelos, sin tocar el motor del selector. Es un diseño modular: en vez de reescribir listas a mano, se crean variables explícitas (cuadráticos, interacciones y dummies de FE) y luego se decide si se incluyen mediante switches.

Hay dos familias de switches:

(i) **Tratamiento** (TR_USE_*). Controlan lo que entra en el `mlogit` del tratamiento. Estos switches afectan directamente los pesos (porque los pesos vienen de las probabilidades predichas del `mlogit`), y por lo tanto afectan ESS, percentiles de pesos y también el balance.

(ii) **Outcome** (Y_USE_*). Controlan lo que entra en el modelo de outcome dentro de `teffects ipwra`. Esto importa para la estimación final del ATET (bloque Sección 22), aunque el selector de diseño se centra principalmente en la parte de pesos/balance.

Dos detalles importantes de implementación:

(1) Se evita usar `i.` y `c.` (factor variables) porque el pipeline trabaja con listas de nombres y chequeos `confirm numeric`. Por eso en Sección 7 se crean las variables explícitas.

(2) Se impone `MAX_SECTOR_FE` para limitar cuántas categorías se dummifican en FE sectoriales. La intuición es evitar que el `mlogit` se rompa por separación o celdas vacías cuando hay muchas categorías con pocos casos.

7. 2.5) Transformaciones y FE (variables explícitas)

Código (líneas 200–341)

```
* 2.5) Transformaciones y FE (creación de variables explícitas)
*=====
* -- Cuadráticos --
capture confirm numeric variable ln_tpo_mean
if !_rc {
    capture drop ln_tpo_sq
    gen double ln_tpo_sq = ln_tpo_mean^2
}
capture confirm numeric variable edad
if !_rc {
    capture drop edad_sq
    gen double edad_sq = edad^2
}
```

```

* -- Interacciones --
capture confirm numeric variable indice_redes
if !_rc {
    capture confirm numeric variable exp_dummy
    if !_rc {
        capture drop redesXexp
        gen double redesXexp = indice_redes*exp_dummy
    }
    capture confirm numeric variable ln_tpo_mean
    if !_rc {
        capture drop redesXsize
        gen double redesXsize = indice_redes*ln_tpo_mean
    }
}

* -- Dummies por ola (anio) --
global FE_WAVE_VARS ""
capture confirm numeric variable anio
if !_rc {
    capture drop wave_*
    quietly tab anio, gen(wave_)
    quietly ds wave_*, has(type numeric)
    global FE_WAVE_VARS 'r(varlist)'
}

* -- Dummies sectoriales (industry_id) --
* WARNING: si hay demasiadas categorías, el mlogit puede fallar por separación/
colinealidad.
/*
global FE_SECTOR_VARS ""
capture confirm numeric variable industry_id
if !_rc {
    quietly levelsof industry_id, local(_levs_ind)
    local _K : word count '_levs_ind'
    if ('_K'<=${MAX_SECTOR_FE}) {
        capture drop ind_*
        quietly tab industry_id, gen(ind_)
        quietly ds ind_*, has(type numeric)
        global FE_SECTOR_VARS 'r(varlist)'
    }
    else {
        di as error "industry_id tiene '_K' categorías (>${MAX_SECTOR_FE}). Omito
dummies sectoriales. (Sugerencia: agrupar sectores.)"
    }
}

```

```

*/
* -- Dummies macro-sector (macro_sector) --
gen byte macro_sector = 3    // 1=Manufacturing, 2=Services, 3=Other/missing

* TU definición:
replace macro_sector = 1 if inlist(industry_id, 6)
replace macro_sector = 2 if inlist(industry_id, 2,3,4,5,7,8,11)
replace macro_sector = 3 if missing(industry_id) | inlist(industry_id, 1,9,10)

label define macro_sector 1 "Manufacturing" 2 "Services" 3 "Other/missing", replace
label values macro_sector macro_sector

drop if macro_sector==3

*/
global FE_SECTOR_VARS ""
capture confirm numeric variable macro_sector
if !_rc {
    quietly levelsof macro_sector, local(_levs_ind)
    local _K : word count `_levs_ind'
    if (`_K'<=${MAX_SECTOR_FE}) {
        capture drop mc_*
        quietly tab macro_sector, gen(mc_)
        quietly ds mc_*, has(type numeric)
        global FE_SECTOR_VARS `r(varlist)'
    }
    else {
        di as error "macro_sector tiene `_K' categorías (>${MAX_SECTOR_FE})."
    }
}

tab anio macro_sector if b1_9==1

* -- Construir listas "listas para enchufar" en tratamiento/outcome --
* (Estas son SOLO listas de nombres. La selección L1_ vs contemporánea la hace
    build_xlist.)
global EXTRA_TREAT ""
global EXTRA_OUTCOME ""

if (${TR_USE_SIZE_SQ}==1)    global EXTRA_TREAT    "${EXTRA_TREAT} ln_tpo_sq"
if (${TR_USE_AGE_SQ}==1)    global EXTRA_TREAT    "${EXTRA_TREAT} edad_sq"
if (${TR_USE_INT_RXEXP}==1) global EXTRA_TREAT    "${EXTRA_TREAT} redesXexp"
if (${TR_USE_INT_RXSIZE}==1) global EXTRA_TREAT    "${EXTRA_TREAT} redesXsize"

if (${Y_USE_SIZE_SQ}==1)    global EXTRA_OUTCOME "${EXTRA_OUTCOME} ln_tpo_sq"
if (${Y_USE_AGE_SQ}==1)    global EXTRA_OUTCOME "${EXTRA_OUTCOME} edad_sq"
if (${Y_USE_INT_RXEXP}==1) global EXTRA_OUTCOME "${EXTRA_OUTCOME} redesXexp"

```

```

if (${Y_USE_INT_RXSIZE}==1) global EXTRA_OUTCOME "${EXTRA_OUTCOME} redesXsize"

* FE listas por ecuación (se agregan ‘tal cual’, siempre contemporáneas)
global FE_TREAT_VARS ""
global FE_OUTCOME_VARS ""

if (${TR_USE_FE_WAVE}==1) global FE_TREAT_VARS "${FE_TREAT_VARS} ${
    FE_WAVE_VARS}"
if (${TR_USE_FE_SECTOR}==1) global FE_TREAT_VARS "${FE_TREAT_VARS} ${
    FE_SECTOR_VARS}"

if (${Y_USE_FE_WAVE}==1) global FE_OUTCOME_VARS "${FE_OUTCOME_VARS} ${
    FE_WAVE_VARS}"
if (${Y_USE_FE_SECTOR}==1) global FE_OUTCOME_VARS "${FE_OUTCOME_VARS} ${
    FE_SECTOR_VARS}"

* Limpieza de espacios
local __tmp1 "${EXTRA_TREAT}"
local __tmp1 : list retokenize __tmp1
global EXTRA_TREAT "__tmp1"
local __tmp2 "${EXTRA_OUTCOME}"
local __tmp2 : list retokenize __tmp2
global EXTRA_OUTCOME "__tmp2"
local __tmp3 "${FE_TREAT_VARS}"
local __tmp3 : list retokenize __tmp3
global FE_TREAT_VARS "__tmp3"
local __tmp4 "${FE_OUTCOME_VARS}"
local __tmp4 : list retokenize __tmp4
global FE_OUTCOME_VARS "__tmp4"

if (${DEBUG_XSETS}==1) {
    di as text "--- X switches ---"
    di as text "EXTRA_TREAT = |${EXTRA_TREAT}|"
    di as text "EXTRA_OUTCOME = |${EXTRA_OUTCOME}|"
    di as text "FE_TREAT_VARS = |${FE_TREAT_VARS}|"
    di as text "FE_OUTCOME_VARS = |${FE_OUTCOME_VARS}|"
}

*=====

```

Explicación

Este bloque crea variables auxiliares que luego pueden entrar en las ecuaciones según los switches:

Cuadráticos. `size\_sq` y `edad\_sq` se construyen como cuadrados de tamaño y edad. La idea es capturar no linealidades (p.ej. efectos marginales decrecientes) sin depender de factor

notation.

Interacciones. `redesXexp` y `redesXsize` crean interacción entre redes y exportación / tamaño. Esto permite que el efecto de redes en la selección al tratamiento dependa del perfil exportador o del tamaño.

Fixed effects (FE). Se crean dummies de ola (`anio\_d*`) y de sector (`mc\_*`). Notá que en este archivo, el sector se construye como `macro\_sector` a partir de `macro\_sector\_ine`, y luego se descarta `macro\_sector==3`. Es decir, el análisis se restringe a dos macro-sectores (1 y 2) y se usa esa partición como FE.

El bloque también arma dos macros globales cruciales: `FE\_TREAT\_VARS` y `FE\_OUTCOME\_VARS`. Estos son los “contenedores” de dummies que se agregan automáticamente a las listas `X` según Sección 6. El diseño evita tocar manualmente las specs cuando prendés/apagás FE.

En términos de navegación del pipeline: este bloque solo *crea* variables. La decisión de si entran en una spec ocurre en Sección 12 y en el armado de `BASEVARS` / `Y\_BASEVARS` en Sección 8–Sección 9.

8. 5) Covariables candidatas del tratamiento

Código (líneas 342–359)

```
* 5) Covariables candidatas (GLOBAL) + L1_ (si USE_LAGS=1)
*=====
* Base (mínimo) -- ajustá nombres a tu base
global BASEVARS_CORE edad capital_extranjero share_prof_tecn exp_dummy ln_tpo_mean
    indice_redes
* BASEVARS (tratamiento) = core + FE según switches
global BASEVARS ${BASEVARS_CORE} ${FE_TREAT_VARS} ${EXTRA_TREAT}
* Opcionales (agregás 1 por vez) -- ajustá a tu gusto
global ADD1 // indice_redes
global ADD2 // indice_obstaculos // ojo, no usar si el GLOBAL USE LAG esta en 0.
global ADD3 apoyo_publico
global ADD4 grupo_economico

* "exporta_mean" puede ser sustituto de "exp_dummy"
* "share_stem" puede ser sustituto de "share_prof_tecn"
* "ln_ing_ventas_mean" puede ser sustituo de "ln_tpo_mean"
* "indice_obstaculos" solo usar rezagado

*=====
```

Explicación

Acá se define el set base de covariables candidatas para el modelo de tratamiento. La macro `BASEVARS\_CORE` es el núcleo mínimo (edad, capital extranjero, share de profesionales/técnicos,

dummy exportadora, productividad/ventas en log, índice de redes).

Luego se define BASEVARS como:

$$\text{BASEVARS} = \text{BASEVARS_CORE} + \text{FE_TREAT_VARS} + \text{EXTRA_TREAT}.$$

Esto es importante porque la construcción de specs S1-S5 se hace agregando “paquetes” (ADD1-ADD4) encima de ese núcleo. La ventaja es que podés cambiar el núcleo (o encender FE) sin reescribir cada spec.

También se documentan posibles sustitutos conceptuales de algunas covariables (p.ej. `exporta\_mean` en vez de `exp\_dummy`). El código no los usa automáticamente: es una guía para que el usuario edite las macros si quiere.

9. 5.b) Covariables del modelo de outcome

Código (líneas 360–394)

```
* 5.b) Covariables del modelo de outcome (GLOBAL)
*=====
* Por defecto, el outcome model usa las mismas X que el treatment model (Xbase de
  la spec).
* Si querés cambiar eso sin romper el flujo:
*   - YMODEL_SAME_AS_TREAT = 1  -> X_y = Xbase  (+ Y_EXTRAVARS)
*   - YMODEL_SAME_AS_TREAT = 0  -> X_y se arma con Y_BASEVARS + Y_ADDVARS (+
    Y_EXTRAVARS)

global YMODEL_SAME_AS_TREAT 0
* Y_BASEVARS (outcome) = core + FE según switches (si YMODEL_SAME_AS_TREAT==0)
global Y_BASEVARS ${BASEVARS_CORE} ${FE_OUTCOME_VARS} ${EXTRA_OUTCOME}
global Y_ADDVARS  indice_redes
global Y_EXTRAVARS // ln_inten_invpo_mean // ojo, no usar si el GLOBAL USE LAG esta
  en 0.

* Por las dudas: fuerza delimitador "normal"
#delimit cr

if $USE_LAGS == 1 {

    * Crear L1_ para TODAS las covariables “laggeables” (no incluye FE dummies)
    local Xlist ${BASEVARS_CORE} ${ADD1} ${ADD2} ${ADD3} ${ADD4} ${Y_ADDVARS} ${
Y_EXTRAVARS} ${EXTRA_TREAT} ${EXTRA_OUTCOME}
    local Xlist : list uniq Xlist

    foreach v of local Xlist {
        * Solo si existe y es numérica
```

```

        capture confirm numeric variable `v'
        if !_rc {
            capture drop L1_`v'
            gen double L1_`v' = L1.`v'
        }
    }
}

*=====

```

Explicación

Este bloque define el set de covariables para el modelo de outcome (Y). La idea central es permitir dos regímenes:

Regla A: outcome igual a treatment. Si `YMODEL\_SAME\_AS\_TREAT=1`, el outcome model usa el mismo X de la spec (posiblemente con extras).

Regla B: outcome armado aparte. Si `YMODEL\_SAME\_AS\_TREAT=0`, el outcome model se arma con `Y\_BASEVARS + Y\_ADDVARS (+ Y\_EXTRAVARS)`. En esta versión, `YMODEL\_SAME\_AS\_TREAT` está en 0 y por eso el bloque 8 usa `Y\_BASEVARS`.

Además, si `USE\_LAGS==1`, se crean rezagos `L1\_*` para todas las covariables “laggeables”. El código lo hace de forma robusta: primero arma una lista única `Xlist`, chequea que cada variable exista y sea numérica, y recién ahí genera `L1\_var = L1.var`. Esto evita que el pipeline se rompa si alguna covariable opcional no existe en la base.

La conexión con el resto del do-file es clara: Sección 12 y Sección 13 construyen `X\_S1-X\_S5` para el tratamiento; el outcome model se decide recién cuando se corre la estimación final en Sección 22.

10. 3) Restricción central y construcción del outcome

Código (líneas 395–410)

```

* 3) Restricción central: innovadores + outcome observado
*=====
keep if b1_9==1

cap drop Y_cond
gen byte Y_cond = inlist(resul_mejora_ambiente,1,2) if !missing(
    resul_mejora_ambiente)
drop if missing(Y_cond)

label var Y_cond "Y (cond): mejora amb media/alta entre innovadores"

```



```

di as text "-----"
di as text "Diagnóstico (muestra b1_9==1 con Y_cond observado)"
tab Y_cond, missing
di as text "-----"

*=====

```

Explicación

Este bloque fija la **muestra analítica** y construye el outcome binario.

Primero, restringe a `b1\_9==1`. Por el nombre y el comentario del bloque, `b1\_9` es una condición central (probablemente “innovadores”).

Segundo, construye `Y_cond` a partir de `resul\_mejora\_ambiente`: se define como 1 si el resultado es 1 o 2 (interpretado como “media/alta” según la etiqueta), y se elimina todo lo que quede missing. Luego muestra un diagnóstico con `tab Y_cond`.

Este bloque es importante metodológicamente: toda la evaluación de diseño posterior (balance, pesos) se hace *condicional a esta muestra*. Es decir, si cambiás la definición de `Y_cond` o la restricción `b1\_9==1`, las métricas cambian porque cambia el universo sobre el cual se estima el `mlogit` y se calculan pesos.

11. 4) Construcción del tratamiento y auxiliares de organización

Código (líneas 411–451)

```

* 4) Tratamiento prod/proc (estado) + organización (auxiliares)
*=====

* Tratamiento base (ignora org en definición)
cap drop D_pp_ignore
gen byte D_pp_ignore = .
replace D_pp_ignore = 0 if innova_producto==0 & innova_proceso==0
replace D_pp_ignore = 1 if innova_producto==1 & innova_proceso==0
replace D_pp_ignore = 2 if innova_producto==0 & innova_proceso==1
replace D_pp_ignore = 3 if innova_producto==1 & innova_proceso==1

label define Dpp 0 "No prod/proc" 1 "Solo producto" 2 "Solo proceso" 3 "Ambas",
    replace
label values D_pp_ignore Dpp
label var D_pp_ignore "D (estado): prod/proc/ambas (ignora org)"

* Auxiliares de org (para controles)
cap drop org1 org_miss

```

```

gen byte org1 = (innova_organiz_comerc==1) if !missing(innova_organiz_comerc)
replace org1 = 0 if missing(org1)
gen byte org_miss = missing(innova_organiz_comerc)

label var org1      "Org/comerc=1 (missing->0)"
label var org_miss "Org/comerc missing"

* Modo estricto: excluir org==1 y org missing
cap drop D_pp_strict
gen byte D_pp_strict = D_pp_ignore
replace D_pp_strict = . if innova_organiz_comerc==1
replace D_pp_strict = . if missing(innova_organiz_comerc)
label values D_pp_strict Dpp
label var D_pp_strict "D (estado): prod/proc/ambas (excluye org1 y org_miss)"

di as text "Distribución D_pp_ignore:"
tab D_pp_ignore, missing
di as text "Distribución D_pp_strict (incluye missings por exclusión):"
tab D_pp_strict, missing

```

Explicación

Se define el tratamiento multivaluado de cuatro estados, construido a partir de dos dummies: `innova\_producto` e `innova\_proceso`. El mapeo es:

0: no producto ni proceso; 1: solo producto; 2: solo proceso; 3: ambas.

Esto se guarda en `D_pp_ignore` y se etiqueta con `label define Dpp`.

Luego se crean auxiliares para innovación organizacional/comercial: `org1` (indicador de 1) y `org_miss` (missing). Esto habilita un *modo* donde la organización entra como control en el `mlogit` (sin redefinir el tratamiento).

Finalmente se arma `D_pp_strict`: es igual a `D_pp_ignore` pero se setea missing cuando `innova\_organiz\_comerc==1` o es missing. Este tratamiento “estricto” implementa una restricción de universo: solo se consideran firmas con org/comerc observada y en cero. Por eso, más adelante, el modo estricto hace `drop if missing(D_pp_strict)`.

La intuición de los dos modos (que se usa en Sección 17) es:

IGNORE_ORG: no condiciona el universo por organización; si se desea, la controla en X.

STRICT_AVOID_ORG: directamente elimina casos con org/comerc=1 o missing y redefine el tratamiento sobre el subconjunto restante.

12. Programa build_xlist: armar listas de covariables con/sin rezagos

Código (líneas 452–552)

```
* (b) Helper: construir lista X usando L1_ cuando existe, si no usar contemporánea
*=====
* FIX ROBUSTO: build_xlist
*=====
cap program drop build_xlist
program define build_xlist, rclass
    version 19.0
    syntax , BASE(string asis) [ADD(string asis)]

    * 1) Normalizar whitespace Unicode -> espacio (incluye NBSP, tabs, CR/LF, etc.)
    local base_clean = ustrregexra("`base'", "[[:space:]]+", " ")
    local add_clean = ustrregexra("`add'", "[[:space:]]+", " ")

    local base_clean = itrim(trim("`base_clean'"))
    local add_clean = itrim(trim("`add_clean'"))

    if ("`base_clean'")=="" {
        di as error "build_xlist: BASE() vacío (no hay covariables base)."
        return local xlist ""
        return local skipped ""
        exit 198
    }

    * 2) Retokenizar como listas
    local basevars : list retokenize base_clean
    local addvars : list retokenize add_clean

    * 3) Elegir L1_ si existe y tiene algún non-missing; si no, contemporánea.
    local x ""
    local skipped ""

    foreach v of local basevars {
        local chosen ""

        * Solo numéricas
        capture confirm numeric variable `v'
        if (_rc) {
            local skipped "`skipped' `v'(non_numeric_or_missing_var)'"
            continue
        }

        if ($USE_LAGS==1) {
```

```

        capture confirm numeric variable L1_`v'
        if (!_rc) {
            quietly count if !missing(L1_`v')
            if (r(N)>0) local chosen "L1_`v'"
        }
    }

    if ("`chosen'""=="") {
        quietly count if !missing(`v')
        if (r(N)>0) local chosen "`v'"
    }

    if ("`chosen'""!="") local x "`x' `chosen'"
    else local skipped "`skipped' `v'(all_missing_in_sample)"
}

foreach v of local addvars {
    if ("`v'""=="") continue
    local chosen ""

    capture confirm numeric variable `v'
    if (_rc) {
        local skipped "`skipped' `v'(non_numeric_or_missing_var)"
        continue
    }

    if ($USE_LAGS==1) {
        capture confirm numeric variable L1_`v'
        if (!_rc) {
            quietly count if !missing(L1_`v')
            if (r(N)>0) local chosen "L1_`v'"
        }
    }

    if ("`chosen'""=="") {
        quietly count if !missing(`v')
        if (r(N)>0) local chosen "`v'"
    }

    if ("`chosen'""!="") local x "`x' `chosen'"
    else local skipped "`skipped' `v'(all_missing_in_sample)"
}

local x : list retokenize x

if ("`x'""=="") {
    di as error "build_xlist: NO quedó ninguna covariable usable (todas
    inexistentes/no numéricas/todo missing)."
}

```

```

        di as error "BASE limpio: |'base_clean'|"
        di as error "ADD limpio: |'add_clean'|"
        di as error "SKIPPED: 'skipped'"
        return local xlist ""
        return local skipped "'skipped'"
        exit 498
    }

    return local xlist "x"
    return local skipped "'skipped'"
end

```

Explicación

`build_xlist` es un helper crítico: toma una lista base (`base()`) y una lista adicional (`add()`) y devuelve `r(xlist)` ya “limpia”, consistente con el régimen de rezagos y con la existencia de variables en la base.

La lógica interna se puede leer en tres pasos:

(1) Sanitización y tokenización. El programa limpia comillas, colapsa espacios múltiples y aplica `retokenize`. Esto evita bugs típicos de Stata cuando una macro trae espacios dobles o comillas.

(2) Construcción de una lista candidata. Une `base` y `add`, elimina duplicados, y deja una única lista ordenada de nombres.

(3) Validación. Para cada nombre: (i) confirma que la variable existe; (ii) confirma que es numérica (requisito para `mlogit` y para métricas de balance); (iii) si `USE_LAGS==1`, reemplaza cada variable por su versión `L1\_var` siempre que exista. Si la versión rezagada no existe, la omite y lo registra en `r(skipped)`.

El resultado final se expone en dos retornos: `r(xlist)` (la lista usable) y `r(skipped)` (variables omitidas). Esto es muy útil para depurar por qué una spec queda vacía o por qué faltan covariables esperadas.

En el flujo general, `build_xlist` es el motor que permite definir `S1-S5` en Sección 13 sin preocuparse por rezagos o por variables opcionales que pueden no existir.

13. 5.c) Construcción de especificaciones S1–S5

Código (líneas 553–601)

```

macro list BASEVARS
di as result "BASEVARS=|${BASEVARS}|"

*=====
*=====

```

```

* 5.c) Construir specs candidatas (X del tratamiento)
*=====
* Nota: estas specs SOLO afectan el mlogit del tratamiento + métricas de balance.
* El outcome model se arma aparte (YMODEL_* + Y_* + EXTRA_OUTCOME).

if (${DEBUG_XSETS}==1) {
    di as text "BASEVARS (treat) = |${BASEVARS}|"
    di as text "EXTRA_TREAT      = |${EXTRA_TREAT}|"
    di as text "FE_TREAT_VARS   = |${FE_TREAT_VARS}|"
}

quietly build_xlist, base(${BASEVARS}) add("")
global X_S1 "r(xlist)"

quietly build_xlist, base(${BASEVARS}) add(${ADD1})
global X_S2 "r(xlist)"

quietly build_xlist, base(${BASEVARS}) add(${ADD1} ${ADD2})
global X_S3 "r(xlist)"

quietly build_xlist, base(${BASEVARS}) add(${ADD1} ${ADD2} ${ADD3})
global X_S4 "r(xlist)"

quietly build_xlist, base(${BASEVARS}) add(${ADD1} ${ADD2} ${ADD3} ${ADD4})
global X_S5 "r(xlist)"

di as text "-----"
di as text "Specs candidatas (treatment X):"
di as text "S1: ${X_S1}"
di as text "S2: ${X_S2}"
di as text "S3: ${X_S3}"
di as text "S4: ${X_S4}"
di as text "S5: ${X_S5}"
di as text "USE_LAGS = ${USE_LAGS}"
di as text "INCLUDE_ORG_CONTROLS = ${INCLUDE_ORG_CONTROLS} | ORG_IN_YMODEL = ${ORG_IN_YMODEL}"
di as text "-----"

tempfile base_data
save 'base_data', replace

*=====
* 6) Programa: evalúa diseño (mlogit) y métricas por tlevel
*=====

```

Explicación

Este bloque construye las especificaciones candidatas S1-S5 para el modelo de selección del tratamiento.

La regla es incremental: todas parten de BASEVARS y van agregando paquetes opcionales:

S1: base; S2: base + ADD1; S3: base + ADD1+ADD2; S4: base + ADD1+ADD2+ADD3; S5: base + ADD1+ADD2+ADD3+ADD4.

Cada vez se llama a `build_xlist` para obtener una lista consistente con `USE_LAGS` y con la existencia de variables.

Dos ideas metodológicas que el propio código deja explícitas:

- (1) Estas specs afectan el `mlogit` del tratamiento y por lo tanto los pesos y el balance (lo que el selector rankea). No determinan, por sí solas, el outcome model de `ipwra`.
- (2) Se guarda una copia del dataset como `base_data`. Eso sirve para “volver” a la muestra base cuando se hacen loops más adelante (Sección 17 y Sección 22), evitando acumulación accidental de `drop` y `keep` entre iteraciones.

14. Programa `eval_design`: proxy IPW manual (`mlogit + w=p_t/p_0`)

Código (líneas 602–750)

```
cap program drop eval_design
program define eval_design
    version 19.0
    syntax , HANDLE(name) DVAR(name) XLIST(string asis) MODE(string) SPEC(string)

    *-- Guardar dataset original (para dejar todo como estaba al salir)
    tempfile __orig __work
    quietly save `__orig', replace

    *-- Trabajar en una copia (sin preserve)
    keep if inlist(`dvar',0,1,2,3)

    * -- sanitizar xlist: sacar comillas y normalizar espacios --
    local xvars ``xlist''
    local xvars : subinstr local xvars `""' "", all
    local xvars = ustrregexra("`xvars'", "[[:space:]]+", " ")
    local xvars = itrim(trim("`xvars'"))

    * -- limpiar covariables 100% missing / inexistentes --
    local xkeep ""
    foreach x of local xvars {
        capture confirm variable `x'
```

```

    if (_rc) continue

    quietly count if !missing('x')
    if (r(N)>0) local xkeep "'xkeep' 'x'"
    else di as error "Mode='mode' Spec='spec': covariable 'x' es TODO missing
-> la saco."
}

local xkeep : list retokenize xkeep
if ("xkeep"=="") {
    di as error "Mode='mode' Spec='spec': X quedó vacío (no hay covariables con
datos) -> salteo."
    use '__orig', clear
    exit
}

* Listwise deletion sobre X usable
egen byte __missX = rowmiss('xkeep')
drop if __missX>0
drop __missX

quietly count
local N_used = r(N)
if ('N_used' < ${MIN_N_USED}) {
    di as error "Muy poca muestra para Mode='mode' Spec='spec' (N='N_used').
Salteo."
    use '__orig', clear
    exit
}

local xvars "'xkeep'"

* Estimar mlogit del tratamiento
quietly capture mlogit 'dvar' 'xvars', baseoutcome(0) nolog
if (_rc) {
    di as error "FALLÓ mlogit para Mode='mode' Spec='spec' (separación/
colinealidad/celdas vacías)."
    use '__orig', clear
    exit
}

tempvar p0 p1 p2 p3
quietly predict double 'p0', outcome(0)
quietly predict double 'p1', outcome(1)
quietly predict double 'p2', outcome(2)
quietly predict double 'p3', outcome(3)

* Guardar dataset de trabajo (ya con p0..p3)

```



```

quietly save '__work', replace

* check
* (debug desactivado)
* summ 'p0' if 'dvar'==0, detail
* summ 'p't' if 'dvar'==0, detail
* summ 'w' if 'dvar'==0, detail
* di as result "N0='N0' sumw='sumw' sumw2='sumw2' ESS0='ESS0'"
* Loop por comparación (t vs 0) sin preserve
forvalues t = 1/3 {
    use '__work', clear
    keep if inlist('dvar',0,'t')

    tempvar w w2 bad_w
    gen double 'w' = .
    replace 'w' = 1 if 'dvar'=='t'
    replace 'w' = ('p't'/'p0') if 'dvar'==0

    gen byte 'bad_w' = ('dvar'==0 & ('p0'<=${PO_FLOOR} | missing('w') | 'w'>${
W_CEIL}))
    drop if 'bad_w'==1

    quietly count if 'dvar'=='t'
    local N1 = r(N)
    quietly count if 'dvar'==0
    local N0 = r(N)

* ESS0
quietly summarize 'w' if 'dvar'==0, meanonly
local sumw = r(sum)
gen double 'w2' = 'w'^2 if 'dvar'==0
quietly summarize 'w2' if 'dvar'==0, meanonly
local sumw2 = r(sum)
local ESS0 = .
if ('sumw2' > 0 & 'sumw2' < .) local ESS0 = ('sumw'^2)/'sumw2'

* Concentración de pesos
quietly summarize 'w' if 'dvar'==0, detail
local w_p95 = r(p95)
local w_p99 = r(p99)
local w_max = r(max)

* Balance
local maxSMD = 0
local sumAbs = 0
local k = 0
local maxLogVR = 0

```

```

foreach x of local xvars {
    quietly summarize 'x' if 'dvar'=='t'
    local mt = r(mean)
    local vt = r(Var)

    quietly summarize 'x' [aw='w'] if 'dvar'==0
    local mc = r(mean)
    local vc = r(Var)

    local sdpool = .
    if ('vt' < . & 'vc' < .) local sdpool = sqrt(('vt' + 'vc')/2)

    if ('sdpool' > 0 & 'sdpool' < .) {
        local smd = ('mt' - 'mc')/'sdpool'
        local abssmd = abs('smd')
        if ('abssmd' > 'maxSMD') local maxSMD = 'abssmd'
        local sumAbs = 'sumAbs' + 'abssmd'
        local ++k

        if ('vt' > 0 & 'vc' > 0 & 'vt' < . & 'vc' < .) {
            local logvr = abs(log('vc'/'vt'))
            if ('logvr' > 'maxLogVR') local maxLogVR = 'logvr'
        }
    }

    local meanSMD = .
    if ('k' > 0) local meanSMD = 'sumAbs'/'k'

    post 'handle' ("mode") ("spec") ('t') ('N1') ('N0') ('ESS0') ///
        ('maxSMD') ('meanSMD') ('maxLogVR') ('w_p95') ('w_p99') ('w_max') ('
N_used')
}

* Volver a como estaba el dataset original
use '__orig', clear
end

```

Explicación

`eval_design` es un evaluador “proxy” que estima un `mlogit` para el tratamiento y, para cada comparación t vs 0, construye pesos de controles como $w = p_t/p_0$. Luego calcula métricas de concentración de pesos y balance.

Qué hace, técnicamente:

Primero limpia la lista `xlist` recibida, elimina covariables inexistentes o 100 % missing, y hace

eliminación listwise (`rowmiss`) para alinear la muestra con lo que en general hace `teffects` cuando faltan covariables.

Luego estima `mlogit D X, baseoutcome(0)` y predice `p0,p1,p2,p3`. Con eso, por cada `t` crea `w` y descarta controles con `p0<=P0_FLOOR`, pesos missing o `w>W_CEIL`.

Después calcula:

ESS0 en controles: $(\sum w)^2 / \sum w^2$.

Concentración de pesos: `p95,p99,max` de w entre controles.

Balance: para cada covariable, calcula medias y varianzas en tratados (sin pesos) y en controles ponderados (con `[aw=w]`), y obtiene SMD y $|\log(VR)|$. Reporta `maxSMD`, `meanSMD` y `maxLogVR`.

Intuición y límites:

Este evaluador sirve como diagnóstico rápido, pero no replica todas las decisiones de trimming usadas por `teffects ipwra`. Por ejemplo, en esta versión el piso de `p0` se aplica solo a controles (no a tratados), y no hay trimming explícito por cuantiles salvo el techo duro. Por eso el pipeline se apoya más en `eval_design_teb` (benchmark) y, si se activa, en `eval_design_tew` (más comparable a `teffects`).

15. Programa `eval_design_tew`: diseño con trimming y pesos tipo `teffects`

Código (líneas 751–994)

```
*****
* 6B) Programa: evalúa diseño con pesos ATET tipo teffects (comparables con ipwra)
*   Idea: para cada comparación (t vs 0), los pesos de controles son w = p_t/p_0
*   (y los tratados t tienen peso 1). Esto replica la lógica de ATET por IPW
*   que usa teffects; además, normalizamos w en controles para que mean(w|D=0)=1
*   cuando computamos percentiles/max de pesos (comparabilidad entre specs).
*****
cap program drop eval_design_tew
program define eval_design_tew
    version 19.0
    syntax , HANDLE(name) DVAR(name) XLIST(string asis) MODE(string) SPEC(string)

    *-- Guardar dataset original (para dejar todo como estaba al salir)
    tempfile __orig __work
    quietly save `__orig', replace
```

```

*-- Trabajar en una copia (sin preserve)
keep if inlist('dvar',0,1,2,3)

* -- sanitizar xlist: sacar comillas y normalizar espacios --
local xvars "'xlist'"
local xvars : subinstr local xvars "" "" , all
local xvars = ustrregexra("xvars", "[[:space:]]+", " ")
local xvars = itrim(trim("xvars"))

* -- limpiar covariables 100% missing / inexistentes --
local xkeep ""
foreach x of local xvars {
    capture confirm variable `x'
    if (_rc) continue

    quietly count if !missing(`x')
    if (r(N)>0) local xkeep "xkeep' `x'"
    else di as error "Mode='mode' Spec='spec': covariable `x' es TODO missing
-> la saco."
}

local xkeep : list retokenize xkeep
if ("xkeep'"=="") {
    di as error "Mode='mode' Spec='spec': X quedó vacío (no hay covariables con
datos) -> salteo."
    use '__orig', clear
    exit
}

* Listwise deletion sobre X usable (alineado con teffects)
egen byte __missX = rowmiss('xkeep')
drop if __missX>0
drop __missX

quietly count
local N_used = r(N)
if ('N_used' < ${MIN_N_USED}) {
    di as error "Muy poca muestra para Mode='mode' Spec='spec' (N='N_used').
Salteo."
    use '__orig', clear
    exit
}

local xvars "xkeep'"

* Estimar mlogit del tratamiento (mismo diseño que la corrida final
multivaluada)
quietly capture mlogit `dvar' `xvars', baseoutcome(0) nolog

```

```

if (_rc) {
    di as error "FALLÓ mlogit para Mode='mode' Spec='spec' (separación/
colinealidad/celdas vacías).\"
    use '__orig', clear
    exit
}

tempvar p0 p1 p2 p3
quietly predict double 'p0', outcome(0)
quietly predict double 'p1', outcome(1)
quietly predict double 'p2', outcome(2)
quietly predict double 'p3', outcome(3)

* Guardar dataset de trabajo (ya con p0..p3)
quietly save '__work', replace

local WTRIM = upper("${W_TRIM_STYLE}")
local do_ps_trim = ${PS_TRIM}
local ps_low = ${PS0_LOW}
local ps_high = ${PS0_HIGH}

* Loop por comparación (t vs 0)
forvalues t = 1/3 {
    use '__work', clear
    keep if inlist('dvar',0,'t')

    * Tamaños antes de recortes (para reportar shares)
    quietly count if 'dvar'=='t'
    local N1_init = r(N)
    quietly count if 'dvar'==0
    local N0_init = r(N)

    tempvar w_raw w w2 flag_p0_ctrl flag_p0_tr flag_ps flag_wceil

    * Reglas explícitas de overlap ( $p0 = \Pr(D=0|X)$ )
    gen byte 'flag_p0_ctrl' = ('p0'<=${P0_FLOOR} | missing('p0')) if 'dvar'==0
    gen byte 'flag_p0_tr' = ('p0'<=${P0_TREAT_FLOOR} | missing('p0')) if '
dvar'=='t'

    gen byte 'flag_ps' = 0
    if ('do_ps_trim'==1) {
        replace 'flag_ps' = ('p0'<'ps_low' | 'p0'>'ps_high') if inlist('dvar'
,0,'t')
    }

    quietly count if 'dvar'==0 & 'flag_p0_ctrl'==1
    local drop_p0_ctrl = r(N)
    quietly count if 'dvar'=='t' & 'flag_p0_tr'==1

```

```

local drop_p0_tr    = r(N)
quietly count if 'flag_ps'==1
local drop_ps_all   = r(N)

drop if ('dvar'==0 & 'flag_p0_ctrl'==1) | ('dvar'=='t' & 'flag_p0_tr'==1) |
('flag_ps'==1)

* Pesos tipo ATET(t vs 0):
*   tratados (D=t): 1
*   controles (D=0): p_t / p_0
gen double 'w_raw' = .
replace 'w_raw' = 1 if 'dvar'=='t'
replace 'w_raw' = ('p_t'/'p_0') if 'dvar'==0

* Corte duro de seguridad (explosión numérica / indefinido)
gen byte 'flag_wceil' = ('dvar'==0 & (missing('w_raw') | 'w_raw'<=0 | '
w_raw'>${W_CEIL}))
quietly count if 'flag_wceil'==1
local drop_wceil = r(N)
drop if 'flag_wceil'==1

* -- Trimming por cuantiles (controles): CAP (winsorizar) o DROP (eliminar)
--
local w_cap = .
local share_trim = .
quietly count if 'dvar'==0
local NO_for_trim = r(N)

if ('NO_for_trim' > 0 & ${W_CAP_Q} < .) {
    local pct = 100*${W_CAP_Q}
    quietly centile 'w_raw' if 'dvar'==0, centile('pct')
    local w_cap = r(c_1)

    if ('w_cap' < . & 'w_cap' > 0) {
        quietly count if 'dvar'==0 & 'w_raw' > 'w_cap'
        local n_trim = r(N)
        local share_trim = 'n_trim'/'NO_for_trim'

        if ("WTRIM"=="CAP") {
            replace 'w_raw' = 'w_cap' if 'dvar'==0 & 'w_raw' > 'w_cap'
        }
        else if ("WTRIM"=="DROP") {
            drop if 'dvar'==0 & 'w_raw' > 'w_cap'
        }
    }
}

* Normalizar solo controles para reportar percentiles/máximo comparables:

```

```

* mean(w|D=0)=1 (SMD/VR/ESS son invariantes a escala, pero p99/max no)
gen double 'w' = 'w_raw'
quietly summarize 'w' if 'dvar'==0, meanonly
local mw0 = r(mean)
if ('mw0' > 0 & 'mw0' < .) {
    replace 'w' = 'w_raw'/'mw0' if 'dvar'==0
}

quietly count if 'dvar'=='t'
local N1 = r(N)
quietly count if 'dvar'==0
local N0 = r(N)

* ESS0 (solo controles; invariante a escala)
quietly summarize 'w' if 'dvar'==0, meanonly
local sumw = r(sum)

gen double 'w2' = 'w'^2 if 'dvar'==0
quietly summarize 'w2' if 'dvar'==0, meanonly
local sumw2 = r(sum)

local ESS0 = .
if ('sumw2' > 0 & 'sumw2' < .) local ESS0 = ('sumw'^2)/'sumw2'

* Concentración de pesos (controles; ya normalizados)
quietly summarize 'w' if 'dvar'==0, detail
local w_p95 = r(p95)
local w_p99 = r(p99)
local w_max = r(max)

* Overlap (rangos de p0)
quietly summarize 'p0' if 'dvar'=='t', detail
local p0_t_min = r(min)
local p0_t_max = r(max)
quietly summarize 'p0' if 'dvar'==0, detail
local p0_0_min = r(min)
local p0_0_max = r(max)

* Balance: SMD y ratio de varianzas (en log abs)
local maxSMD = 0
local sumAbs = 0
local k = 0
local maxLogVR = 0

foreach x of local xvars {
    quietly summarize 'x' if 'dvar'=='t'
    local mt = r(mean)
    local vt = r(Var)

```

```

quietly summarize 'x' [iw='w'] if 'dvar'==0
local mc = r(mean)
local vc = r(Var)

local sdpool = .
if ('vt' < . & 'vc' < .) local sdpool = sqrt(('vt' + 'vc')/2)

if ('sdpool' > 0 & 'sdpool' < .) {
    local smd = ('mt' - 'mc')/'sdpool'
    local abssmd = abs('smd')
    if ('abssmd' > 'maxSMD') local maxSMD = 'abssmd'
    local sumAbs = 'sumAbs' + 'abssmd'
    local ++k

    if ('vt' > 0 & 'vc' > 0 & 'vt' < . & 'vc' < .) {
        * VR = vc/vt. Reportamos max|log(VR)| (simétrico: VR=2 y VR=0.5
dan lo mismo).
        local logvr = abs(log('vc'/'vt'))
        if ('logvr' > 'maxLogVR') local maxLogVR = 'logvr'
    }
}

local meanSMD = .
if ('k' > 0) local meanSMD = 'sumAbs'/'k'

local share_drop_p0_ctrl = cond('N0_init'>0, 'drop_p0_ctrl'/'N0_init', .)
local share_drop_p0_tr   = cond('N1_init'>0, 'drop_p0_tr'/'N1_init', .)
local share_drop_ps      = cond(('N0_init'+'N1_init')>0, 'drop_ps_all'/'('
N0_init'+'N1_init'), .)
local share_drop_wceil   = cond(('N0_init'+'N1_init')>0, 'drop_wceil'/'('
N0_init'+'N1_init'), .)

post 'handle' ("mode") ("spec") ('t') ///
    ('N1') ('N0') ('ESS0') ///
    ('maxSMD') ('meanSMD') ('maxLogVR') ///
    ('w_p95') ('w_p99') ('w_max') ('N_used') ///
    ('w_cap') ('share_trim') ('share_drop_p0_ctrl') ('share_drop_p0_tr') ('
share_drop_ps') ('share_drop_wceil') ///
    ('p0_t_min') ('p0_t_max') ('p0_0_min') ('p0_0_max')
}

* Volver a como estaba el dataset original
use '__orig', clear
end

```


Explicación

`eval_design_tew` es un evaluador más cercano a la lógica de `teffects` para ATET, porque explicita reglas de overlap y además normaliza pesos para que los percentiles sean comparables entre specs.

La estructura general (igual que en `eval_design`) es: limpiar `X`, estimar `mlogit`, predecir `p0..p3`, y luego iterar por `t=1,2,3`. La diferencia está en el detalle de los recortes y en cómo reporta métricas:

- 1) **Overlap por `p0`**. Define banderas separadas para controles (`P0_FLOOR`) y tratados (`P0_TREAT_FLOOR`). Además, si `PS_TRIM=1`, elimina observaciones con `p0` fuera del rango `[PS0_LOW, PS0_HIGH]`. Se reportan shares de drop por cada regla.
- 2) **Pesos ATET y techo duro**. Construye `w_raw` y descarta controles con pesos no positivos, missing o mayores a `W_CEIL`.
- 3) **Trimming por cuantiles**. Calcula `w_cap` como el cuantil `W_CAP_Q` de `w_raw` en controles. Luego aplica CAP (winsorización) o DROP según `W_TRIM_STYLE`. Reporta `share_trim`.
- 4) **Normalización para reportar percentiles**. Define `w = w_raw / mean(w_raw|D=0)` en controles, de manera que $E[w \mid D = 0] = 1$. Importante: esto *no cambia* SMD/VR/ESS (son invariantes a escala), pero sí hace que `p99/max` sean comparables entre specs.
- 5) **Balance con `[iw=w]`**. Calcula medias/varianzas de controles con `[iw=w]` (importancia). Luego calcula `maxSMD`, `meanSMD` y `maxLogVR`.

Este evaluador es particularmente útil si querés que las gráficas de pesos y las métricas numéricas cuenten la misma historia, porque explicita exactamente qué observaciones fueron filtradas por cada regla.

16. Programa `eval_design_teb`: benchmark multinomial y métricas de balance

Código (líneas 995–1528)

```
*-----
*-----
* 4) Evaluación "benchmark" (MULTINOMIAL) para balance del diseño IPWRA
*
* IDEA (alineada con el manual .tex):
*   - Como D es multi-valued (0,1,2,3), el modelo correcto de propensión es
*     MULTINOMIAL (mlogit).
*   - Estimamos una sola vez  $p_j(X) = \Pr(D=j|X)$  con baseoutcome(0) usando X del
*     treatment model (XTR).
*   - Para cada contraste t vs 0 (t=1,2,3), armamos pesos ATET en controles:
*      $w0_i = p_t(X_i) / p_0(X_i)$  si  $D_i=0$ 
```

```

*          w1_i = 1                      si D_i=t
*      (Luego normalizamos w0 para que  $E[w_0|D=0]=1$ ; esto NO cambia SMD ni el ATET
*      tipo Hájek.)
*      - Calculamos métricas de balance sobre el set UNION(XTR, XY) para reflejar el
*      diseño IPWRA,
*      sin depender de tebalance (que además estaba corriendo en binario y te dejaba
*      todo vacío).
*
* OPCIONAL (apagado por defecto):
*      - Si RUN_BIN_BENCH=1, se puede habilitar un fallback binario con teffects para
*      casos extremos,
*      pero se imprime un aviso grande en consola porque es una APROXIMACIÓN.
*-----

capture program drop eval_design_teb
program define eval_design_teb
    version 19.0
    syntax , HANDLE(name) DVAR(name) XTR(string asis) XY(string asis) MODE(string)
    SPEC(string)

    tempfile __orig __work
    quietly save `__orig', replace

    keep if inlist(`dvar',0,1,2,3)

    *-----
    * 4.1) Sanitizar listas + filtrar covariables "vacías"
    *-----
    local xtrvars "'xtr'"
    local xyvars  "'xy'"

    local xtrvars : subinstr local xtrvars '""' "" , all
    local xyvars  : subinstr local xyvars  '""' "" , all

    local xtrvars = ustrregexra("`xtrvars'", "[[:space:]]+", " ")
    local xyvars  = ustrregexra("`xyvars'",  "[[:space:]]+", " ")
    local xtrvars = itrim(trim("`xtrvars'"))
    local xyvars  = itrim(trim("`xyvars'"))

    local xtr_keep ""
    foreach x of local xtrvars {
        capture confirm variable `x'
        if (_rc) continue
        quietly count if !missing(`x')
        if (r(N)>0) local xtr_keep "`xtr_keep' `x'"
        else di as error "Mode='mode' Spec='spec': covariable (XTR) `x' es TODO
missing -> la saco."
    }

```

```

local xy_keep ""
foreach x of local xyvars {
    capture confirm variable `x'
    if (_rc) continue
    quietly count if !missing(`x')
    if (r(N)>0) local xy_keep "`xy_keep' `x'"
    else di as error "Mode=`mode' Spec=`spec': covariable (XY) `x' es TODO
missing -> la saco."
}

local xtr_keep : list retokenize xtr_keep
local xy_keep : list retokenize xy_keep

if ("`xtr_keep'=="") {
    di as error "Mode=`mode' Spec=`spec': XTR quedó vacío -> salteo benchmark."
    forvalues t=1/3 {
        quietly count if `dvar'==`t'
        local N1 = r(N)
        quietly count if `dvar'==0
        local N0 = r(N)
        post `handle' ("`mode'") ("`spec'") (`t') (`N1') (`N0') (.) (.) (.) (.)
        (.) (.) (.) ///
        (`N1'+`N0') (.) (.) (.) (.) (.) (.) (.) (0) ("") ("")
    }
    use `__orig', clear
    exit
}

* UNION(XTR,XY) para balance (IPWRA permite X distintos en TM y YM)
local x_all "`xtr_keep' `xy_keep'"
local x_all : list uniq x_all

* Listwise deletion en X_all (alineado con lo que haría teffects en práctica)
egen byte __missX = rowmiss(`x_all')
drop if __missX>0
drop __missX

quietly count
local N_used0 = r(N)
if (`N_used0' < ${MIN_N_USED}) {
    di as error "Muy poca muestra para Mode=`mode' Spec=`spec' (N=`N_used0')".
    Salteo benchmark."
    forvalues t=1/3 {
        quietly count if `dvar'==`t'
        local N1 = r(N)
        quietly count if `dvar'==0
        local N0 = r(N)

```

```

        post 'handle' ("mode") ("spec") ('t') ('N1') ('N0') (.) (.) (.) (.)
        (.) (.) (.) (.) (.) (.) (.) (.) (0) ("") ("")
    }
    use '__orig', clear
    exit
}

*-----
* 4.2) Propensiones MULTINOMIALES (mlogit) - tratamiento model
*-----
quietly capture mlogit 'dvar' 'xtr_keep', baseoutcome(0) nolog
if (_rc) {
    di as error "Mode='mode' Spec='spec': FALLÓ mlogit (multinomial) del
    tratamiento."

    *-----
    * Fallback binario (logit por contraste t vs 0) - APROXIMACIÓN
    *-----
    if (${RUN_BIN_BENCH}==1) {
        noi di as error "    >>> RUN_BIN_BENCH=1: usando fallback BINARIO (logit
        ) por contraste t vs 0."
        noi di as error "        OJO: esto NO es el multinomial completo; se usa
        solo para completar el ranking cuando mlogit falla."

        local WTRIM = upper("${W_TRIM_STYLE}")
        local do_ps_trim = ${PS_TRIM}
        local ps_low = ${PS0_LOW}
        local ps_high = ${PS0_HIGH}

        forvalues t=1/3 {

            preserve
            keep if inlist('dvar',0,'t')

            tempvar Dbin ps p0 w_raw w w2 bad_ctrl bad_tr bad_ps bad_wceil

            gen byte 'Dbin' = ('dvar'=='t')

            quietly capture logit 'Dbin' 'xtr_keep'
            if (_rc) {
                noi di as error "    >>> Fallback binario: FALLÓ logit para
                t='t' (Mode='mode' Spec='spec')."
                quietly count if 'dvar'=='t'
                local N1 = r(N)
                quietly count if 'dvar'==0
                local N0 = r(N)
            }
        }
    }
}

```

```

        post 'handle' ("mode") ("spec") ('t') ('N1') ('NO') (.)
        (.) (.) (.) (.) (.) (.) ///
        ('N1'+ 'NO') (.) (.) (.) (.) (.) (.) (0) ("" ) ("" )
        restore
        continue
    }

    quietly predict double 'ps' if e(sample), pr
    gen double 'p0' = 1 - 'ps'

    quietly count if 'dvar'=='t'
    local N1_init = r(N)
    quietly count if 'dvar'==0
    local NO_init = r(N)
    local N_all_init = 'N1_init' + 'NO_init'

    gen byte 'bad_ctrl' = ('dvar'==0 & ('p0'<=${PO_FLOOR} | missing
('p0'))
    gen byte 'bad_tr' = ('dvar'=='t' & ('p0'<=${PO_TREAT_FLOOR} |
missing('p0'))

    gen byte 'bad_ps' = 0
    if ('do_ps_trim'==1) {
        replace 'bad_ps' = ('p0'<'ps_low' | 'p0'>'ps_high') if
inlist('dvar',0,'t')
    }

    drop if 'bad_ctrl' | 'bad_tr' | 'bad_ps'

    gen double 'w_raw' = .
    replace 'w_raw' = 1 if 'dvar'=='t'
    replace 'w_raw' = ('ps'/'p0') if 'dvar'==0

    gen byte 'bad_wceil' = ('dvar'==0 & (missing('w_raw') | 'w_raw'
<=0 | 'w_raw'>${W_CEIL}))
    drop if 'bad_wceil'

    * Trimming por cuantil (solo controles)
    local w_cap = .
    quietly count if 'dvar'==0
    local NO_for_trim = r(N)

    if ('NO_for_trim' > 0 & ${W_CAP_Q} < .) {
        local pct = 100*${W_CAP_Q}
        quietly centile 'w_raw' if 'dvar'==0, centile('pct')
        local w_cap = r(c_1)

        if ('w_cap' < . & 'w_cap' > 0) {

```

```

        if ("WTRIM"=="CAP") {
            replace 'w_raw' = 'w_cap' if 'dvar'==0 & 'w_raw' >
'w_cap'
        }
        else if ("WTRIM"=="DROP") {
            drop if 'dvar'==0 & 'w_raw' > 'w_cap'
        }
    }
}

* Normalizar controles a media 1 (no cambia SMD/ATET tipo Hájek
)

gen double 'w' = 'w_raw'
quietly summarize 'w' if 'dvar'==0, meanonly
local mw0 = r(mean)
if ('mw0'>0 & 'mw0'<.) replace 'w' = 'w_raw'/'mw0' if 'dvar'==0

quietly count if 'dvar'=='t'
local N1 = r(N)
quietly count if 'dvar'==0 & 'w' < .
local N0 = r(N)
local N_used = 'N1' + 'N0'

local share_os_all = .
local share_os_1 = .
local share_os_0 = .
if ('N_all_init'>0) local share_os_all = 1 - ('N_used'/'
N_all_init')

if ('N1_init'>0)    local share_os_1    = 1 - ('N1'/'N1_init')
if ('N0_init'>0)    local share_os_0    = 1 - ('N0'/'N0_init')

quietly summarize 'w' if 'dvar'==0, detail
local w_p95 = r(p95)
local w_p99 = r(p99)
local w_max = r(max)

quietly summarize 'w' if 'dvar'==0, meanonly
local sumw = r(sum)
gen double 'w2' = 'w'^2 if 'dvar'==0
quietly summarize 'w2' if 'dvar'==0, meanonly
local sumw2 = r(sum)

local ESS0 = .
if ('sumw2'>0 & 'sumw2'<.) local ESS0 = ('sumw'^2)/'sumw2'

quietly summarize 'p0' if 'dvar'=='t', detail
local p0_t_min = r(min)
local p0_t_max = r(max)

```

```

quietly summarize 'p0' if 'dvar'==0, detail
local p0_0_min = r(min)
local p0_0_max = r(max)

* Balance sobre UNION(XTR,XY) que ya está en 'x_all'
local xbal "'x_all'"

local k=0
local sumAbs=0
local maxAbs=-1
local maxAbsVar ""
local maxLogVR=-1
local maxLogVRVar ""
local has_vr=0

foreach x of local xbal {
    capture confirm variable 'x'
    if (_rc) continue

    quietly summarize 'x' if 'dvar'=='t'
    local m1=r(mean)
    local v1=r(Var)

    * Controles: media ponderada
    quietly summarize 'x' [iw='w'] if 'dvar'==0
    local m0=r(mean)

    * Controles: varianza ponderada (alineada con tebalance)
    *  $v0 = \sum_i w_i (x_i - \bar{x}_w)^2 / (\sum_i w_i - 1)$ 
    tempvar __dev __wdev
    quietly gen double '__dev' = ('x' - 'm0')^2 if 'dvar'==0 &
'w' < . & !missing('x')
    quietly gen double '__wdev' = 'w' * '__dev' if 'dvar'==0 &
'w' < . & !missing('x')

    quietly summarize 'w' if 'dvar'==0 & 'w' < .
    local sumw = r(sum)

    quietly summarize '__wdev' if 'dvar'==0 & !missing('__wdev')
)

local sumwdev = r(sum)

local v0 = .
if ('sumw' > 1) local v0 = 'sumwdev' / ('sumw' - 1)

quietly drop '__dev' '__wdev'

```

```

        local denom = sqrt(('v1'+v0)/2)
        if ('denom'>0 & 'denom'<.) {
            local smd = ('m1' - 'm0')/'denom'
            local a = abs('smd')
            local k = 'k' + 1
            local sumAbs = 'sumAbs' + 'a'
            if ('a' > 'maxAbs') {
                local maxAbs = 'a'
                local maxAbsVar "'x'"
            }
        }

        if ('v0'>0 & 'v1'>0 & 'v0'<. & 'v1'<.) {
            local lv = abs(log('v1'/'v0'))
            if ('lv' > 'maxLogVR') {
                local maxLogVR = 'lv'
                local maxLogVRVar "'x'"
            }

            local has_vr = 1
        }

    }

    local maxSMD_w = .
    local meanSMD_w = .
    local maxLogVarRatio_w = .
    local parse_ok = 0

    if ('k' > 0) {
        local maxSMD_w = 'maxAbs'
        local meanSMD_w = 'sumAbs'/'k'
        local parse_ok = 1
    }

    if ('has_vr'==1) local maxLogVarRatio_w = 'maxLogVR'

    local col_wsd "'maxAbsVar'"
    local col_wvr "'maxLogVRVar'"

    post 'handle' ("mode") ("spec") ('t') ('N1') ('N0') ('ESS0'
) ///

        ('maxSMD_w') ('meanSMD_w') ('maxLogVarRatio_w') ///
        ('w_p95') ('w_p99') ('w_max') ('N_used') ///
        ('p0_t_min') ('p0_t_max') ('p0_0_min') ('p0_0_max') ///
        ('share_os_all') ('share_os_1') ('share_os_0') ('parse_ok')
    ("col_wsd") ("col_wvr")

        restore
    }
    use '__orig', clear
    exit

```



```

    }
    else {
        forvalues t=1/3 {
            quietly count if `dvar'==`t'
            local N1 = r(N)
            quietly count if `dvar'==0
            local N0 = r(N)
            post `handle' ("`mode'"') ("`spec'"') (`t') (`N1') (`N0') (.) (.) (.)
            (.) (.) (.) (.) ///
                (`N1'+`N0') (.) (.) (.) (.) (.) (.) (.) (0) ("") ("")
        }
        use `__orig', clear
        exit
    }
}

tempvar p0 p1 p2 p3
quietly predict double `p0', outcome(0)
quietly predict double `p1', outcome(1)
quietly predict double `p2', outcome(2)
quietly predict double `p3', outcome(3)

quietly save `__work', replace

local WTRIM = upper("${W_TRIM_STYLE}")
local do_ps_trim = ${PS_TRIM}
local ps_low = ${PS0_LOW}
local ps_high = ${PS0_HIGH}

*-----
* 4.3) Para cada t vs 0: pesos ATET + métricas (SMD/VR/ESS/pesos/overlap)
*-----
forvalues t=1/3 {
    use `__work', clear
    keep if inlist(`dvar',0,`t')

    quietly count if `dvar'==`t'
    local N1_init = r(N)
    quietly count if `dvar'==0
    local N0_init = r(N)
    local N_all_init = `N1_init' + `N0_init'

    tempvar w_raw w w2 bad_ctrl bad_tr bad_ps bad_wceil
    gen byte `bad_ctrl' = (`dvar'==0 & (`p0'<=${P0_FLOOR} | missing(`p0'))))
    gen byte `bad_tr'   = (`dvar'==`t' & (`p0'<=${P0_TREAT_FLOOR} | missing(`p0'
    )))

    gen byte `bad_ps' = 0

```

```

    if ('do_ps_trim'==1) {
        replace 'bad_ps' = ('p0'<'ps_low' | 'p0'>'ps_high') if inlist('dvar'
,0,'t')
    }

    drop if 'bad_ctrl' | 'bad_tr' | 'bad_ps'

    gen double 'w_raw' = .
    replace 'w_raw' = 1 if 'dvar'=='t'
    replace 'w_raw' = ('p't'/'p0') if 'dvar'==0

    gen byte 'bad_wceil' = ('dvar'==0 & (missing('w_raw') | 'w_raw'<=0 | 'w_raw
'>${W_CEIL}))
    drop if 'bad_wceil'

    * Trimming por cuantil (solo controles)
    local w_cap = .
    quietly count if 'dvar'==0
    local NO_for_trim = r(N)

    if ('NO_for_trim' > 0 & ${W_CAP_Q} < .) {
        local pct = 100*${W_CAP_Q}
        quietly centile 'w_raw' if 'dvar'==0, centile('pct')
        local w_cap = r(c_1)

        if ('w_cap' < . & 'w_cap' > 0) {
            if ("WTRIM"=="CAP") {
                replace 'w_raw' = 'w_cap' if 'dvar'==0 & 'w_raw' > 'w_cap'
            }
            else if ("WTRIM"=="DROP") {
                drop if 'dvar'==0 & 'w_raw' > 'w_cap'
            }
        }
    }

    * Normalizar controles a media 1 (no cambia SMD/ATET tipo Hájek)
    gen double 'w' = 'w_raw'
    quietly summarize 'w' if 'dvar'==0, meanonly
    local mw0 = r(mean)
    if ('mw0'>0 & 'mw0'<.) replace 'w' = 'w_raw'/'mw0' if 'dvar'==0

    quietly count if 'dvar'=='t'
    local N1 = r(N)
    quietly count if 'dvar'==0 & 'w' < .
    local NO = r(N)
    local N_used = 'N1' + 'NO'

    * Share "excluida" por overlap/trimming (sustituye osample())

```

```

local share_os_all = .
local share_os_1 = .
local share_os_0 = .
if ('N_all_init'>0) local share_os_all = 1 - ('N_used'/'N_all_init')
if ('N1_init'>0)    local share_os_1  = 1 - ('N1'/'N1_init')
if ('N0_init'>0)    local share_os_0  = 1 - ('N0'/'N0_init')

* Estadísticos de pesos en controles
quietly summarize 'w' if 'dvar'==0, detail
local w_p95 = r(p95)
local w_p99 = r(p99)
local w_max = r(max)

* ESS0 en controles: (sum w)^2 / sum(w^2)
quietly summarize 'w' if 'dvar'==0, meanonly
local sumw = r(sum)
gen double 'w2' = 'w'^2 if 'dvar'==0
quietly summarize 'w2' if 'dvar'==0, meanonly
local sumw2 = r(sum)

local ESS0 = .
if ('sumw2'>0 & 'sumw2'<.) local ESS0 = ('sumw'^2)/'sumw2'

* Overlap: rango de p0 en tratados y controles (post-filtros)
quietly summarize 'p0' if 'dvar'=='t', detail
local p0_t_min = r(min)
local p0_t_max = r(max)

quietly summarize 'p0' if 'dvar'==0, detail
local p0_0_min = r(min)
local p0_0_max = r(max)

*-----
* 4.4) Balance: SMD + max|log(VR)| sobre UNION(XTR,XY)
*-----
local xbal "'x_all'"

local k=0
local sumAbs=0
local maxAbs=-1
local maxAbsVar ""
local maxLogVR=-1
local maxLogVRVar ""
local has_vr=0

foreach x of local xbal {
    capture confirm variable 'x'
    if (_rc) continue

```

```

quietly summarize 'x' if 'dvar'=='t'
local m1=r(mean)
local v1=r(Var)

* Controles: media ponderada
quietly summarize 'x' [iw='w'] if 'dvar'==0
local m0=r(mean)

* Controles: varianza ponderada (alineada con tebalance)
*   v0 = sum_i w_i (x_i - xbar_w)^2 / (sum_i w_i - 1)
tempvar __dev __wdev
quietly gen double '__dev' = ('x' - 'm0')^2 if 'dvar'==0 & 'w' < . & !
missing('x')
quietly gen double '__wdev' = 'w' * '__dev' if 'dvar'==0 & 'w' < . & !
missing('x')

quietly summarize 'w' if 'dvar'==0 & 'w' < .
local sumw = r(sum)

quietly summarize '__wdev' if 'dvar'==0 & !missing('__wdev')
local sumwdev = r(sum)

local v0 = .
if ('sumw' > 1) local v0 = 'sumwdev' / ('sumw' - 1)

quietly drop '__dev' '__wdev'

local denom = sqrt(('v1'+v0)/2)
if ('denom'>0 & 'denom'<.) {
    local smd = ('m1' - 'm0')/'denom'
    local a = abs('smd')
    local k = 'k' + 1
    local sumAbs = 'sumAbs' + 'a'
    if ('a' > 'maxAbs') {
        local maxAbs = 'a'
        local maxAbsVar "'x'"
    }
}

if ('v0'>0 & 'v1'>0 & 'v0'<. & 'v1'<.) {
    local lv = abs(log('v1'/'v0')) // equivalente a abs(log(VR))
    if ('lv' > 'maxLogVR') {
        local maxLogVR = 'lv'
        local maxLogVRVar "'x'"
    }
    local has_vr = 1
}

```

```

    }

    local maxSMD_w = .
    local meanSMD_w = .
    local maxLogVarRatio_w = .
    local parse_ok = 0

    if ('k' > 0) {
        local maxSMD_w = 'maxAbs'
        local meanSMD_w = 'sumAbs'/'k'
        local parse_ok = 1
    }

    if ('has_vr'==1) local maxLogVarRatio_w = 'maxLogVR'
    local col_wsd "maxAbsVar"
    local col_wvr "maxLogVRVar"
    post 'handle' ("mode") ("spec") ('t') ('N1') ('N0') ('ESS0') ///
        ('maxSMD_w') ('meanSMD_w') ('maxLogVarRatio_w') ///
        ('w_p95') ('w_p99') ('w_max') ('N_used') ///
        ('p0_t_min') ('p0_t_max') ('p0_0_min') ('p0_0_max') ///
        ('share_os_all') ('share_os_1') ('share_os_0') ('parse_ok') ("col_wsd"
    ") ("col_wvr")
    }

    use '__orig', clear
end

=====
=====

```

Explicación

`eval_design_teb` es el evaluador “benchmark” que el do-file usa por defecto para rankear diseños (`RUN_TEB=1`). La idea es acercarse más al comportamiento final de `teffects ipwra` con tratamiento multivaluado: no solo importa el `mlogit` (pesos), sino también que el outcome model sea estimable y que el balance sea razonable.

La firma del programa separa explícitamente covariables del modelo de tratamiento (`xtr()`) y del modelo de outcome (`xy()`). Esto permite que, si querés, el selector evalúe diseños donde la selección usa un set y el outcome otro (en el pipeline actual, normalmente se usan iguales o casi iguales según modo).

La lógica interna tiene varias etapas:

A) Construcción de una muestra consistente. Se queda con `D in 0..3`, sanitiza `xtr` y `xy`, elimina covariables inexistentes o sin datos, y hace eliminación listwise conjunta para alinear la muestra con la estimación final.

B) Pesos ATET por comparación. Estima `mlogit` del tratamiento y predice `p0..p3`. Para cada `t`, se queda con `t vs 0`, aplica reglas de overlap (piso de `p0` en ambos grupos y, si se activa, trimming por rango de `p0`), aplica techo duro `W_CEIL` y trimming por cuantiles `W_CAP_Q`.

C) Métricas de pesos. Calcula ESS0 y estadísticos de concentración de pesos en controles (`p95/p99/max`), y también reporta `w_cap` y shares de trimming/drops.

D) Métricas de balance. Para cada covariable en `xtr`, calcula: - media y varianza en tratados (sin ponderación); - media y varianza en controles ponderados por `w` (con un cálculo explícito de varianza ponderada, coherente con la idea de `tebalance summarize`).

Luego construye `maxSMD_w` y `meanSMD_w`, y `maxLogVarRatio_w`. Además guarda `col_wsd` y `col_wvr`: los nombres de las covariables que alcanzan el máximo SMD y el máximo VR. Esto es muy valioso para depurar: si una spec falla por balance, sabés exactamente *qué covariable* lo está causando.

E) Soporte/off-support. El programa computa `share_os*` como fracciones asociadas a observaciones fuera del soporte según reglas de recorte (y/o fallas de estimación). En el pipeline, estas métricas luego se usan para comparar diseños entre sí.

En síntesis: este programa consolida en un solo lugar todo lo que el selector considera “calidad de diseño” (muestra usable, overlap, pesos, y balance), y por eso alimenta los datasets `design_long_teb` y `design_summary_teb` que se usan en Sección 21.

17. 7) Ejecución del selector (loops por modo y spec)

Código (líneas 1529–1643)

```
* 7) Ejecutar selector (2 bloques: IPW-proxy + benchmark tebalance)
*=====

tempfile design_long_ipw design_long_teb winners merged_rank
tempname H_ipw H_teb

* -- Bloque A: IPW-proxy (manual) --
postfile 'H_ipw' str20 mode str10 spec int t ///
    double N1 NO ESS0 maxSMD meanSMD maxLogVarRatio ///
        w_p95 w_p99 w_max N_used ///
        w_cap share_trim share_drop_p0_ctrl share_drop_p0_tr share_drop_ps
share_drop_wceil ///
    p0_t_min p0_t_max p0_0_min p0_0_max ///
    using 'design_long_ipw', replace

* -- Bloque B: Benchmark teffects + tebalance (Weighted) --
postfile 'H_teb' str20 mode str10 spec int t ///
    double N1 NO ESS0 maxSMD_w meanSMD_w maxLogVarRatio_w ///
        w_p95 w_p99 w_max N_used ///
```

```

        p0_t_min p0_t_max p0_0_min p0_0_max ///
        share_os_all share_os_1 share_os_0 parse_ok ///
    str32 col_wsd str32 col_wvr ///
    using 'design_long_teb', replace

* -----
* MODO 1: IGNORE_ORG
* -----
preserve
    di as text "=== MODO: IGNORE_ORG ==="

    * Outcome y tratamiento ya fueron contruidos arriba.
    capture confirm variable Y_cond
    if (_rc) {
        di as error "Falta Y_cond. Revisá el BLOQUE 3 (construcción del outcome)."
        exit 111
    }
    capture confirm variable D_pp_ignore
    if (_rc) {
        di as error "Falta D_pp_ignore. Revisá el BLOQUE 4 (construcción del
tratamiento)."
        exit 111
    }

    drop if missing(Y_cond)

    local specs "S1 S2 S3 S4 S5"
    foreach spec of local specs {

        local Xbase "${X_`spec'}"

        * En IGNORE_ORG: opcionalmente agregamos org1 y org_miss como covariables
        local Xtr "'Xbase'"
        local Xy "'Xbase'"

        if $INCLUDE_ORG_CONTROLS==1 {
            local Xtr "'Xbase' org1 org_miss"
            if $ORG_IN_YMODEL==1 {
                local Xy "'Xbase' org1 org_miss"
            }
        }

        if (${RUN_TEW}==1) {
            quietly eval_design_tew, handle('H_ipw') dvar(D_pp_ignore) ///
                xlist("'Xtr'") mode("IGNORE_ORG") spec("'spec'")
        }

        if (${RUN_TEB}==1) {

```

```

        quietly eval_design_teb, handle('H_teb') dvar(D_pp_ignore) ///
            xtr("Xtr'") xy("Xy'") mode("IGNORE_ORG") spec("'spec'")
    }
}
restore

* -----
* MODO 2: STRICT_AVOID_ORG
* -----
preserve
    di as text "=== MODO: STRICT_AVOID_ORG ==="

    capture confirm variable Y_cond
    if (_rc) {
        di as error "Falta Y_cond. Revisá el BLOQUE 3 (construcción del outcome)."
        exit 111
    }
    capture confirm variable D_pp_strict
    if (_rc) {
        di as error "Falta D_pp_strict. Revisá el BLOQUE 4 (construcción del
tratamiento estricto)."
        exit 111
    }

    drop if missing(Y_cond)
    drop if missing(D_pp_strict)

    local specs "S1 S2 S3 S4 S5"
    foreach spec of local specs {

        local Xbase "${X_'spec'}"

        * En STRICT se excluye org==1 y org missing; no metemos org1/org_miss como
        controles.
        if (${RUN_TEW}==1) {
            quietly eval_design_tew, handle('H_ipw') dvar(D_pp_strict) ///
                xlist("Xbase'") mode("STRICT_AVOID_ORG") spec("'spec'")
        }

        if (${RUN_TEB}==1) {
            quietly eval_design_teb, handle('H_teb') dvar(D_pp_strict) ///
                xtr("Xbase'") xy("Xbase'") mode("STRICT_AVOID_ORG") spec("'spec'
")
        }
    }
}
restore

```



```
postclose 'H_ipw'
postclose 'H_teb'
```

```
*=====
*=====
```

Explicación

Este bloque es el “main” del pipeline: prepara archivos de salida temporales, abre `postfile` para ir guardando métricas, y recorre modos y especificaciones.

Hay dos salidas principales:

design_long_teb: una tabla “larga” con una fila por (modo, spec, t).

design_summary_teb: un resumen que agrega sobre `t` para cada (modo, spec) (promedios, máximos, etc.), y que es el insumo directo para el ranking.

El loop implementa exactamente la definición de modos discutida en Sección 11:

Modo IGNORE_ORG. Usa `D_pp_ignore`. Si `INCLUDE_ORG_CONTROLS==1`, agrega `org1` y `org_miss` a `Xtr`; y si además `ORG_IN_YMODEL==1`, también los agrega a `Xy`. Esto permite controlar organización sin redefinir tratamiento.

Modo STRICT_AVOID_ORG. Usa `D_pp_strict` y elimina `missing(D_pp_strict)`, que por construcción incluye `org=1` y `org missing`. En este modo no se agregan covariables de `org`.

En cada `spec`, el bloque decide qué evaluadores correr según `RUN_TEW` y `RUN_TEB`. En la configuración actual, solo corre `eval_design_teb`.

18. 7.1) Exportación de tablas largas

Código (líneas 1644–1674)

```
* 7.1) Exportar tablas largas (controlado por RUN_TEW / RUN_TEB)
*=====
if (${RUN_TEW}==1) {
  use 'design_long_ipw', clear
  save "design_balance_long_ipw.dta", replace

  if (${SHOW_LONG_TABLES}==1) {
    di as text " "
    di as text "--- DESIGN (TEW / IPW-proxy) : primeras filas ---"
    list mode spec t N1 NO ESS0 maxSMD meanSMD maxLogVarRatio w_p99 w_max if _n
    <=16, noobs
  }
}
else {
```

```

    di as text "RUN_TEW=0 -> no se exporta design_balance_long_ipw.dta (proxy IPW
    apagado)."
```

}

```

if (${RUN_TEB}==1) {
    use 'design_long_teb', clear
    save "design_balance_long_teb.dta", replace

    if (${SHOW_LONG_TABLES}==1) {
        di as text " "
        di as text "--- DESIGN (TEB / BENCHMARK MULTINOMIAL) : primeras filas ---"
        list mode spec t N1 NO ESS0 maxSMD_w meanSMD_w maxLogVarRatio_w w_p99 w_max
        share_os_all parse_ok if _n<=16, noobs
    }
}
else {
    di as error "RUN_TEB=0 -> OJO: sin benchmark no hay ranking principal."
}

=====
```

Explicación

Se exportan a Excel las tablas largas (fila por (modo, spec, t)). El objetivo es inspección manual: mirar, por ejemplo, si una spec funciona bien para t=1 pero mal para t=3, o si el modo estricto colapsa en alguna comparación.

El detalle de columnas exportadas está definido en el `postfile` del bloque Sección 17. Ahí queda claro qué métrica viene de qué evaluador (TEB vs TEW).

19. 7.2) Resumen agregado por (modo, spec)

Código (líneas 1675–1733)

```

* 7.2) Agregar resumen por (mode,spec)
=====
tempfile ipw_agg teb_agg merged_teb

*--- (Opcional) TEW agregado ---
if (${RUN_TEW}==1) {
    use 'design_long_ipw', clear
    collapse (min) minN1=N1 minNO=NO minESS0=ESS0 ///
              (max) maxSMD=maxSMD meanSMD=meanSMD maxLogVarRatio=maxLogVarRatio ///
              w_p95=w_p95 w_p99=w_p99 w_max=w_max ///
    , by(mode spec)
```

```

    rename minN1    minN1_ipw
    rename minNO    minNO_ipw
    rename minESS0  minESS0_ipw

    rename maxSMD      maxSMD_ipw
    rename meanSMD     meanSMD_ipw
    rename maxLogVarRatio maxLogVarRatio_ipw
    rename w_p95       w_p95_ipw
    rename w_p99       w_p99_ipw
    rename w_max       w_max_ipw

    save 'ipw_agg', replace
}

*--- TEB benchmark multinomial agregado (este es el que rankeamos) ---
use 'design_long_teb', clear
collapse (min) minN1=N1 minNO=N0 minESS0=ESS0 min_parse_ok=parse_ok ///
          (max) maxSMD_w=maxSMD_w meanSMD_w=meanSMD_w maxLogVarRatio_w=
          maxLogVarRatio_w ///
          w_p95=w_p95 w_p99=w_p99 w_max=w_max ///
          share_os_all=share_os_all share_os_1=share_os_1 share_os_0=
share_os_0 ///
          , by(mode spec)

rename minN1      minN1_teb
rename minNO      minNO_teb
rename minESS0    minESS0_teb
rename min_parse_ok min_parse_ok_teb

rename maxSMD_w      maxSMD_w_teb
rename meanSMD_w     meanSMD_w_teb
rename maxLogVarRatio_w maxLogVarRatio_w_teb
rename w_p95         w_p95_teb
rename w_p99         w_p99_teb
rename w_max         w_max_teb
rename share_os_all  share_os_all_teb
rename share_os_1    share_os_1_teb
rename share_os_0    share_os_0_teb

save 'teb_agg', replace

*--- Merge opcional con TEW ---
use 'teb_agg', clear
if (${RUN_TEW}==1) {
    merge 1:1 mode spec using 'ipw_agg', nogen
}
save 'merged_teb', replace

```

```
*=====
```

Explicación

Este bloque construye y exporta un resumen por (modo, spec) agregando sobre t. La lógica es:

- promedios: meanSMD_w, meanLogVarRatio_w, mean_w_p99, etc.; - máximos: maxSMD_w, maxLogVarRatio_w, max_w_max, etc.; - tamaños y ESS: mean_N0, mean_ESS0, etc.

Este resumen es el objeto que luego se rankea, porque es lo que permite comparar specs con un solo número por métrica. La exportación a Excel (design_summary_by_spec.xlsx) es lo que típicamente mirás para entender por qué una spec gana.

20. 7.3) Checks de consistencia entre TEW y TEB

Código (líneas 1734–1752)

```
* 7.3) Check de consistencia (TEB vs TEW) - solo si RUN_TEW=1
*=====
if (${RUN_TEW}==1) {
  use 'merged_teb', clear

  gen absdiff_maxSMD = abs(maxSMD_w_teb - maxSMD_ipw)
  gen absdiff_meanSMD = abs(meanSMD_w_teb - meanSMD_ipw)
  gen absdiff_logVR = abs(maxLogVarRatio_w_teb - maxLogVarRatio_ipw)

  quietly summarize absdiff_maxSMD absdiff_meanSMD absdiff_logVR, detail

  di as text " "
  di as text "--- Consistencia TEW vs TEB (resumen) ---"
  di as text "absdiff maxSMD:  p50=" %9.4f r(p50) "  p95=" %9.4f r(p95) "  max="
    %9.4f r(max)

  * Nota: si querés ver todo, poné SHOW_LONG_TABLES=1 arriba y mirá las tablas
  largas.
}

*=====
```

Explicación

Si RUN_TEW==1, este bloque compara consistencia de métricas entre el evaluador TEW y el TEB (por ejemplo, maxSMD_w). La motivación es validar que ambas formas de computar balance/pesos cuentan una historia similar; si divergen mucho, es una señal de que el proxy no está alineado con la estimación final.

Como en esta versión RUN_TEW=0, este bloque queda como “infraestructura” para cuando quieras activar TEW.

21. 7.4) Ranking y elección de ganadores

Código (líneas 1753–1854)

```
* 7.4) Ranking y ganadores finales por modo (basado en TEB multinomial)
*   Cascada:
*       (0) soporte mínimo
*       (1) maxSMD <= 0.20
*       (2) var ratio en niveles entre [0.5, 2]  <=> VRsym <= 2
*       (3) entre los que pasan (1)&(2): mayor ESS0, luego pesos menos extremos
*       (4) fallback cuando NO pasa VR: minimizar violación VR (absLogVR), luego
           ESS0, luego pesos
*=====
use 'merged_teb', clear

* -- auxiliares --
tempvar absLogVR VRsym key1 key2
gen double 'absLogVR' = abs(maxLogVarRatio_w_teb)
gen double 'VRsym'    = exp('absLogVR')    // >=1 (niveles, simétrico)

* -- Umbrales locales SOLO para ranking --
local SMD_TOL_RANK 0.20
local VR_TOL_LEVEL 2          // niveles (simétrico: 0.5-2)
local LOGVR_TOL    = ln('VR_TOL_LEVEL')    // 0.693... (referencia; no se usa
directo)

* -- soporte + filtros --
gen okSupport_teb = (min_parse_ok_teb>=1) & (minN1_teb>=${MIN_N1}) & ///
                    (minNO_teb>=${MIN_NO}) & (minESS0_teb>0) if !missing(
min_parse_ok_teb)
replace okSupport_teb = 0 if missing(okSupport_teb)

gen okSMD_teb = (maxSMD_w_teb <= 'SMD_TOL_RANK') if !missing(maxSMD_w_teb)
replace okSMD_teb = 0 if missing(okSMD_teb)

gen okVar_teb = ('VRsym' <= 'VR_TOL_LEVEL') if !missing('VRsym')
replace okVar_teb = 0 if missing(okVar_teb)

* -- grupos --
gen byte rank_group = .
replace rank_group = 1 if okSupport_teb==1 & okSMD_teb==1 & okVar_teb==1
replace rank_group = 2 if okSupport_teb==1 & okSMD_teb==1 & okVar_teb==0
replace rank_group = 3 if okSupport_teb==1 & okSMD_teb==0
replace rank_group = 4 if okSupport_teb==0
```

```

* -- claves de orden --
* Grupo 1: (pasa VR) -> ESS0 desc, luego pesos asc
* Grupo 2: (NO pasa VR) -> minimizar violación VR (absLogVR asc), luego ESS0 desc,
    luego pesos asc
gen double 'key1' = .
replace 'key1' = -minESS0_teb      if rank_group==1
replace 'key1' = 'absLogVR'      if rank_group==2
replace 'key1' = maxSMD_w_teb     if rank_group==3
replace 'key1' = 1e9              if rank_group==4

gen double 'key2' = .
replace 'key2' = w_p99_teb        if rank_group==1
replace 'key2' = -minESS0_teb     if rank_group==2
replace 'key2' = meanSMD_w_teb    if rank_group==3
replace 'key2' = 1e9              if rank_group==4

gsort mode rank_group 'key1' 'key2' w_max_teb w_p95_teb share_os_all_teb
by mode: gen rank_teb = _n

save 'merged_rank', replace

* -- Ganador por modo --
preserve
keep if rank_teb==1
keep mode spec rank_teb rank_group ///
    okSupport_teb okSMD_teb okVar_teb ///
    minN1_teb minN0_teb minESS0_teb min_parse_ok_teb ///
    maxSMD_w_teb meanSMD_w_teb maxLogVarRatio_w_teb ///
    share_os_all_teb share_os_1_teb share_os_0_teb ///
    w_p95_teb w_p99_teb w_max_teb

rename minN1_teb      minN1
rename minN0_teb      minN0
rename minESS0_teb    minESS0
rename min_parse_ok_teb min_parse_ok

rename maxSMD_w_teb    maxSMD_w
rename meanSMD_w_teb   meanSMD_w
rename maxLogVarRatio_w_teb maxLogVarRatio_w

rename share_os_all_teb share_os_all
rename share_os_1_teb   share_os_1
rename share_os_0_teb   share_os_0

rename w_p95_teb w_p95
rename w_p99_teb w_p99

```

```

rename w_max_teb w_max

save 'winners', replace

di as text " "
di as result "Ganadores finales por modo (TEB multinomial) con filtros SMD
<=0.20 y VR in [0.5,2]:"
list mode spec rank_group okSupport_teb okSMD_teb okVar_teb ///
      minN1 minNO minESS0 maxSMD_w maxLogVarRatio_w meanSMD_w share_os_all w_p99
      w_max, noobs

use 'merged_rank', clear // o abrí el .dta que guardaste
list mode spec rank_group okSupport_teb okSMD_teb okVar_teb ///
      minN1_teb minNO_teb minESS0_teb maxSMD_w_teb maxLogVarRatio_w_teb ///
      w_p99_teb w_max_teb, sepby(mode) noobs
by mode: tab rank_group

restore

```

Explicación

Acá ocurre la selección final. El bloque toma `design_summary_teb` (resumen por (modo, spec)) y construye un ranking con reglas explícitas:

- 1) Crea `maxVRsym = exp(maxLogVarRatio_w)`. Esto convierte el máximo de $|\log(\text{VR})|$ en una medida de ratio simétrica en niveles. Por ejemplo, $|\log(2)|$ y $|\log(0,5)|$ producen `maxVRsym=2`.
- 2) Define un flag `pass_balance` que exige simultáneamente: `maxSMD_w <= 0.20` y `maxVRsym <= 2`. Es decir, tolera hasta 0.20 de SMD máximo y hasta VR dentro de $[0.5, 2]$.
- 3) Ordena por: (i) pasar balance primero (los que pasan se priorizan), (ii) minimizar `maxSMD_w`, (iii) minimizar `maxVRsym`, (iv) minimizar `mean_w_p99`, (v) minimizar `mean_w_max`.

La intuición es bien estándar en selección de diseño: primero se exige balance; dentro de los que pasan, se prefieren los que tienen mejor balance y pesos menos concentrados.

- 4) Genera `winners`: la primera spec por modo luego de ordenar.

Este bloque es el que define, de facto, la “función objetivo” del selector. Si en algún momento querés cambiar el criterio (p.ej. ser más estricto con SMD, o penalizar más `share_os`), este es el lugar.

22. 8) Estimación final ATET y gráficos (overlap + hist de pesos)

Código (líneas 1855–2062)

```

* 8) (Opcional) Correr ATET con la spec ganadora + gráficos overlap/pesos
*=====

log using "C:\Users\Equipo\OneDrive\Desktop\proyecto\4Medio ambiente- innovacion\3.
Estrategia empirica\3. Modelos\5Resultado-innovacion - estrategia causal\ATET.
log", replace

if (${RUN_ATET}==1) {

    * Volver a la muestra base
    use 'base_data', clear
    tab anio macro_sector if b1_9==1

    * Abrir ganadoras
    preserve
        use 'winners', clear
        levelsof mode, local(modes)
    restore

    foreach mm of local modes {

        * Tomar la primera spec de ese modo
        preserve
            use 'winners', clear
            keep if mode=="mm"
            keep in 1
            local best_spec "'=spec[1]'"
        restore

        di as text " "
        di as text "=====
        di as text "MOD0 = 'mm' | Spec ganadora = 'best_spec'"
        di as text "=====

        * Definir Dvar según modo
        local Dvar ""
        if ("mm"=="IGNORE_ORG") local Dvar "D_pp_ignore"
        if ("mm"=="STRICT_AVOID_ORG") local Dvar "D_pp_strict"

        * Definir Xbase según spec
        local Xbase ""
        if ("best_spec"=="S1") local Xbase "${X_S1}"
        if ("best_spec"=="S2") local Xbase "${X_S2}"
        if ("best_spec"=="S3") local Xbase "${X_S3}"
        if ("best_spec"=="S4") local Xbase "${X_S4}"
    }
}

```



```

if ("best_spec"=="S5") local Xbase "${X_S5}"

* Treatment-X y Outcome-X (pueden diferir si ORG_IN_YMODEL=0)
local X_tr "Xbase"

* Extras de tratamiento se incluyen vía BASEVARS (ver switches TR_* arriba)

* Outcome-X: por defecto igual al treatment; opcionalmente custom (ver
globals YMODEL_*)
local X_y ""
if (${YMODEL_SAME_AS_TREAT}==1) {
    local X_y "Xbase"
}
else {
    if ("${Y_ADDVARS}"=="") {
        quietly build_xlist, base(${Y_BASEVARS}) add("")
    }
    else {
        quietly build_xlist, base(${Y_BASEVARS}) add(${Y_ADDVARS})
    }
    local X_y "r(xlist)"
}

* Extras para el outcome (en niveles; build_xlist elige L1_ si corresponde)
* Robusto: si por alguna razón ninguna extra es usable, no rompas todo el
do-file.
if ("${Y_EXTRAVARS}"!="") {
    capture quietly build_xlist, base(${Y_EXTRAVARS}) add("")
    if (!_rc) local X_y "X_y' r(xlist)"
}

* Extras opcionales (cuadráticos/interacciones) para outcome
if ("${EXTRA_OUTCOME}"!="") {
    capture quietly build_xlist, base(${EXTRA_OUTCOME}) add("")
    if (!_rc) local X_y "X_y' r(xlist)"
}

if ("mm"=="IGNORE_ORG" & ${INCLUDE_ORG_CONTROLS}==1) {
    local X_tr "X_tr' org1 org_miss"
    if (${ORG_IN_YMODEL}==1) local X_y "X_y' org1 org_miss"
}

* Deduplicar por si hay solapes (evita "variable ... appears more than once")
local X_tr : list uniq X_tr
local X_tr : list retokenize X_tr
local X_y : list uniq X_y
local X_y : list retokenize X_y

```

```

preserve
  * Muestra correspondiente
  keep if inlist('Dvar',0,1,2,3)

  * Drop por missings en variables usadas (alineado con teffects)
  foreach x of local X_tr {
    drop if missing('x')
  }
  foreach x of local X_y {
    drop if missing('x')
  }
  drop if missing(Y_cond)

  * ATET (multivaluado: reporta 1vs0, 2vs0, 3vs0)
  teffects ipwra (Y_cond 'X_y', logit) ('Dvar' 'X_tr', mlogit), ///
    atet control(0) vce(cluster correlativo) osample(os_'mm')

  estimates store ATET_'mm'_'best_spec'
  tebalance summarize

  *--- Overlap: densidad de pscore por tlevel ---
  quietly mlogit 'Dvar' 'X_tr', baseoutcome(0) nolog
  cap drop p0 p1 p2 p3
  predict double p0, outcome(0)
  predict double p1, outcome(1)
  predict double p2, outcome(2)
  predict double p3, outcome(3)

  set scheme s2mono

  local WTRIM = upper("${W_TRIM_STYLE}")
  local do_ps_trim = ${PS_TRIM}
  local ps_low = ${PS0_LOW}
  local ps_high = ${PS0_HIGH}

  forvalues t=1/3 {

    *-----
    * Muestra y pesos EXACTAMENTE como en el ranking
    * (p0 floors + (opcional) PS trimming + wceil + (CAP/DROP) +
normalización)
    *-----
    tempvar __use __w_raw __w __bad_ctrl __bad_tr __bad_ps __bad_wceil
    gen byte '__use' = inlist('Dvar',0,'t')

    gen byte '__bad_ctrl' = ('Dvar'==0 & ('__use'==1) & (p0<=${
PO_FLOOR} | missing(p0)))

```

```

gen byte '__bad_tr' = ('Dvar'=='t' & ('__use'==1) & (p0<=${
PO_TREAT_FLOOR} | missing(p0)))

gen byte '__bad_ps' = 0
if ('do_ps_trim'==1) {
  replace '__bad_ps' = (p0<'ps_low' | p0>'ps_high') if '__use'==1
}

replace '__use' = 0 if '__bad_ctrl' | '__bad_tr' | '__bad_ps'

gen double '__w_raw' = .
replace '__w_raw' = 1 if 'Dvar'=='t' & '__use'==1
replace '__w_raw' = (p't'/p0) if 'Dvar'==0 & '__use'==1

gen byte '__bad_wceil' = ('Dvar'==0 & '__use'==1 & (missing('__w_raw') | '__w_raw'<=0 | '__w_raw'>${W_CEIL}))
replace '__use' = 0 if '__bad_wceil'

* Trimming por cuantil (solo controles, sobre la muestra "usada")
local w_cap = .
quietly count if 'Dvar'==0 & '__use'==1
local NO_for_trim = r(N)

if ('NO_for_trim' > 0 & ${W_CAP_Q} < .) {
  local pct = 100*${W_CAP_Q}
  quietly centile '__w_raw' if 'Dvar'==0 & '__use'==1, centile('
pct')

  local w_cap = r(c_1)

  if ('w_cap' < . & 'w_cap' > 0) {
    if ("WTRIM"=="CAP") {
      replace '__w_raw' = 'w_cap' if 'Dvar'==0 & '__use'==1 &
'__w_raw' > 'w_cap'
    }
    else if ("WTRIM"=="DROP") {
      replace '__use' = 0 if 'Dvar'==0 & '__use'==1 & '
__w_raw' > 'w_cap'
    }
  }
}

* Normalizar controles a media 1 (Hájek): w = w_raw / mean(w_raw|D
=0,usados)
gen double '__w' = '__w_raw'
quietly summarize '__w' if 'Dvar'==0 & '__use'==1, meanonly
local mw0 = r(mean)
if ('mw0'>0 & 'mw0'<.) replace '__w' = '__w_raw'/'mw0' if 'Dvar'==0
& '__use'==1

```

```

        replace `__w' = . if `__use'==0

        *-----
        * Overlap (densidad) y pesos (hist), usando la misma muestra "usada
"
        *-----
        twoway ///
            (kdensity p`t' if `Dvar'==0 & `__use'==1, lpattern(solid))
        ///
            (kdensity p`t' if `Dvar'==`t' & `__use'==1, lpattern(dash)),
        ///
            legend(order(1 "Controles (0)" 2 "Tratados (`t')")) ///
            title("Overlap: Pr(D=`t') | Mode=`mm' Spec=`best_spec'") ///
            xtitle("Probabilidad predicha") ytitle("Densidad")

        graph export "${outdir}\overlap_`mm'_`best_spec'_t`t'.png", replace

        histogram `__w' if `Dvar'==0 & `__use'==1 & !missing(`__w'),
percent ///
            title("Pesos t=`t' | trim=`WTRIM' q=${W_CAP_Q} | Mode=`mm'")
        ///
            xtitle("peso w (normalizado)") ytitle("%")

        graph export "${outdir}\weights_`mm'_`best_spec'_t`t'.png", replace

        drop `__use' `__w_raw' `__w' `__bad_ctrl' `__bad_tr' `__bad_ps' `
__bad_wceil'
        }
        restore
    }
}

di as text "Listo. Resultados en: ${outdir}"

log close

```

Explicación

Este bloque es opcional (RUN_ATET) y corre la estimación final con `teffects ipwra` usando la(s) especificación(es) ganadora(s), además de producir diagnósticos gráficos coherentes con las reglas de trimming del selector.

La lógica es:

- (1) Vuelve a la muestra base (`use base_data`).
- (2) Abre la base `winners` y obtiene la(s) spec ganadora(s) por modo. Para cada modo toma

la primera fila (la de mejor ranking).

(3) Reconstruye `Xtr` y `Xy` de forma consistente con el modo: - En `IGNORE_ORG`, opcionalmente agrega `org1 org_miss` según los switches. - En `STRICT_AVOID_ORG`, usa `D_pp_strict` y elimina missing.

(4) Para cada `t=1,2,3` corre `teffects ipwra` en el subconjunto `D in \{0,t\}` con `atet control(0)` y `vce(cluster correlativo)`.

(5) Genera **overlap plots** con `tebalance overlap` (guardando nombres de archivos por `modo/spec/t`).

(6) Genera **histogramas de pesos** para controles, pero (y esto es lo valioso) aplica exactamente las mismas reglas de filtrado que el selector: - piso de `p0` en controles (`P0_FLOOR`); - techo duro de pesos (`W_CEIL`); - trimming por cuantil `W_CAP_Q` con estilo `W_TRIM_STYLE`; - normalización de pesos para reportar distribución comparable ($E[w \mid D = 0] = 1$).

Esto asegura que “lo visual y lo numérico cuentan la misma historia”: los histogramas reflejan los pesos efectivamente usados en el ranking, no pesos crudos sin recortes.

Nota práctica: el `log using` y algunos paths de salida están hardcodeados con una ruta absoluta específica. Funciona si esa ruta existe, pero si querés portabilidad total conviene reemplazarla por `${outdir}` y construir los nombres de archivo desde ahí.

23. Resumen ejecutivo del do-file

En términos funcionales, el do-file hace lo siguiente:

(1) Carga y prepara la base de panel (`xtset correlativo anio, delta(3)`), construye transformaciones (cuadráticos, interacciones) y fixed effects en forma de dummies explícitas.

(2) Define un outcome binario `Y_cond` y un tratamiento multivaluado con cuatro estados (`D=0,1,2,3`), además de dos regímenes de tratamiento (`ignore vs strict`) para manejar la innovación organizacional.

(3) Define un conjunto de especificaciones candidatas `S1-S5` para el modelo de selección del tratamiento (el `mlogit`), usando el helper `build_xlist` para manejar existencia de variables y rezagos.

(4) Recorre, para cada modo y spec, tres comparaciones binarias (`t=1,2,3 vs 0`) y calcula métricas de: overlap/soporte, concentración de pesos IPW (incluyendo `ESS0`) y balance de covariables (SMD y ratios de varianzas). En la configuración actual, estas métricas provienen del evaluador benchmark `eval_design_teb`.

(5) Agrega resultados por (`modo, spec`), exporta tablas a Excel y elige ganadores mediante un ranking que prioriza: pasar umbrales de balance, y luego minimizar desbalance y concentración de pesos.

(6) Opcionalmente (`RUN_ATET`), corre `teffects ipwra` con la(s) spec(s) ganadora(s) y genera gráficos diagnósticos (overlap y distribución de pesos) aplicando las mismas reglas de trimming

usadas para rankear.

El output final del pipeline es una recomendación reproducible de especificación (**winner** por modo) respaldada por evidencia cuantitativa (tablas de balance/pesos) y, si se activa el módulo final, evidencia gráfica coherente con esas mismas reglas.